Refinement and Type Difference in SKJS

Abstract

We formalize the internal type representation used by SKJS and study two related type transformations: (1) predicate-driven *refinement* (control-flow narrowing), and (2) the type-difference operation (diff) used during type inference. We state and prove soundness properties for these transformations, while explicitly documenting the intended sources of incompleteness.

1 Definitions Index

Symbol	Defined In
Tags	Definition 1
$(\mathsf{Payload}_k)_{k\in\mathsf{Tags}}, \mathsf{Val}, \mathsf{rtag}$	Definition 2
SType	Definition 3
tag	Definition 4
A0	Lemma 5
A1	Lemma 6
$\mathsf{STypes}, \mathsf{tags}(\cdot)$	Definition 9
$\mathsf{STC}(\cdot,\cdot)$	Definition 49
TInst	Definition 13
TInsts	Definition 14
TParam	Definition 15
Clause	Definition 16
$\mathsf{Clauses}$	Definition 17
TType	Definition 18
refine (predicate-driven refinement)	Definition 22
Predicate-soundness	Definition 24
diff (soundness P1/P2)	Definition 44

2 Runtime Model and Semantic Interface

Definition 1 (Runtime Tags)**.** *Let* Tags *be the set of runtime tags (the coarse runtime kind of a value). In the current implementation, tags are represented as integers, so one may take* $\mathsf{Tags} = \{0, 1, \ldots, 10\}$.

Definition 2 (Runtime Values and Tags)**.** *Assume runtime values carry their tag as part of the representation. We fix a family of payload sets* $(\mathsf{Payload}_k)_{k\in\mathsf{Tags}}$ *and define*

$$\mathsf{Val} \triangleq \{\, (k, p) \mid k \in \mathsf{Tags} \ \wedge \ p \in \mathsf{Payload}_k \,\}.$$

Define the runtime tag function $\mathsf{rtag} : \mathsf{Val} \to \mathsf{Tags}$ *as the first projection:*

$$\mathsf{rtag}((k, p)) = k.$$

Concrete payloads (for this document). *We make primitive payloads concrete and leave structured payloads abstract:*

$$\mathsf{Payload}_0 = \{\star\} \qquad \mathsf{Payload}_1 = \{\star\} \qquad \mathsf{Payload}_2 = \{\star\}$$
$$\mathsf{Payload}_3 = \{\mathsf{true}, \mathsf{false}\} \qquad \mathsf{Payload}_4 = \mathbb{R} \qquad \mathsf{Payload}_5 = \mathsf{String}$$
$$\mathsf{Payload}_6 = \mathbb{Z} \qquad \mathsf{Payload}_{10} = \mathsf{String} \times \mathsf{Payload}'_{10}.$$

Here $\mathsf{Payload}'_{10}$ *is an arbitrary set (the constructor payload). For the remaining tags* $k \in \{7, 8, 9\}$ *we leave* $\mathsf{Payload}_k$ *as arbitrary sets.*

Definition 3 (Simple Types)**.** *Let* SType *be the set of* simple types*, i.e. the atomic type forms that have a single runtime tag. Formally,* SType *is the smallest set generated by the following constructors:*

- *primitive units:* `undefined`*,* `void`*,* `null`
- *primitive values with optional finite literal-set payload:*
  - `boolean`(B) *where* $B \subseteq_{\text{fin}} \{\mathsf{true}, \mathsf{false}\}$ *or* $B = \top$*,*
  - `number`(N) *where* $N \subseteq_{\text{fin}} \mathbb{R}$ *or* $N = \top$*,*
  - `string`(S) *where* $S \subseteq_{\text{fin}} \mathsf{String}$ *or* $S = \top$*,*
  - `bigint`(Z) *where* $Z \subseteq_{\text{fin}} \mathbb{Z}$ *or* $Z = \top$*.*
- *structured atoms:* `tuple`$(\ldots)$*,* `Promise`$(\ldots)$*,* `object`$(\ldots)$*,* `algebraic`$(\ldots)$*. These include tagged unions/ADTs with a runtime discriminant.*

We write these constructors suggestively; most internal parameters are left abstract. However, for control-flow refinement we will require a concrete semantic interface for object shapes (fields and index signatures); see Definition 7.

Notation (source-level literal types). *We write* `boolean` *for* `boolean`$(\top)$*, and similarly for* `number`*,* `string`*,* `bigint`*. We write* `true` *for* `boolean`$(\{\mathsf{true}\})$ *and* `false` *for* `boolean`$(\{\mathsf{false}\})$*. We write a numeric literal* n *for* `number`$(\{n\})$*, and a string literal "*x*" for* `string`$(\{x\})$*. More generally, a union like* `1 | 2` *is represented as* `number`$(\{1, 2\})$*, i.e. a single* SType *alternative at the numeric tag. We use these abbreviations uniformly, including under* $\mathsf{tag}(\cdot)$ *and* $[\![\cdot]\!]$*. For example,* $[\![$`number`$]\!]$ *means* $[\![$`number`$(\top)]\!]$*, and* $\mathsf{tag}($`1`$)$ *means* $\mathsf{tag}($`number`$(\{1\}))$*.*

Definition 4 (Simple Types and Tags)**.** *The function* $\mathsf{tag} : \mathsf{SType} \to \mathsf{Tags}$ *assigns tags:*

$$\mathsf{tag}(\texttt{undefined}) = 0 \qquad \mathsf{tag}(\texttt{void}) = 1$$
$$\mathsf{tag}(\texttt{null}) = 2 \qquad \mathsf{tag}(\texttt{boolean}(\cdot)) = 3$$
$$\mathsf{tag}(\texttt{number}(\cdot)) = 4 \qquad \mathsf{tag}(\texttt{string}(\cdot)) = 5$$
$$\mathsf{tag}(\texttt{bigint}(\cdot)) = 6 \qquad \mathsf{tag}(\texttt{tuple}(\ldots)) = 7$$
$$\mathsf{tag}(\texttt{Promise}(\ldots)) = 8 \qquad \mathsf{tag}(\texttt{object}(\ldots)) = 9$$
$$\mathsf{tag}(\texttt{algebraic}(\ldots)) = 10$$

The semantic interpretation $[\![\cdot]\!] : \mathsf{SType} \to \mathcal{P}(\mathsf{Val})$ *maps each simple type to its set of inhabitants. We define* $[\![\cdot]\!]$ *concretely for primitive units and primitive-value payloads (including singleton-literal notation), and define the remaining structured cases at tag level (with the* `object` *and* `algebraic` *cases further constrained in Definitions 7 and 8):*

$$[\![\texttt{undefined}]\!] \triangleq \{(0, \star)\} \qquad [\![\texttt{void}]\!] \triangleq \{(1, \star)\} \qquad [\![\texttt{null}]\!] \triangleq \{(2, \star)\}$$

$$[\![\texttt{boolean}(\top)]\!] \triangleq \{(3, b) \mid b \in \{\mathsf{true}, \mathsf{false}\}\} \qquad [\![\texttt{boolean}(B)]\!] \triangleq \{(3, b) \mid b \in B\}$$

$$[\![\texttt{number}(\top)]\!] \triangleq \{(4, n) \mid n \in \mathbb{R}\} \qquad [\![\texttt{number}(N)]\!] \triangleq \{(4, n) \mid n \in N\}$$

$$[\![\texttt{string}(\top)]\!] \triangleq \{(5, x) \mid x \in \mathsf{String}\} \qquad [\![\texttt{string}(S)]\!] \triangleq \{(5, x) \mid x \in S\}$$

$$[\![\texttt{bigint}(\top)]\!] \triangleq \{(6, z) \mid z \in \mathbb{Z}\} \qquad [\![\texttt{bigint}(Z)]\!] \triangleq \{(6, z) \mid z \in Z\}$$

$$[\![\texttt{tuple}(\ldots)]\!] \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 7\}$$

$$[\![\texttt{Promise}(\ldots)]\!] \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 8\}.$$

In the finite-payload cases, B*,* N*,* S*,* Z *denote finite subsets of the respective domains.*

Lemma 5 (**A0** (Runtime Tag Consistency))**.** *The semantic interpretation of simple types agrees with runtime tags:*

$$\forall s \in \mathsf{SType}.\ \forall v \in \mathsf{Val}.\ v \in [\![s]\!] \implies \mathsf{rtag}(v) = \mathsf{tag}(s).$$

Proof. By case analysis on s. For primitive units and primitive-value payloads, the definition of $[\![s]\!]$ contains only values of the form $(\mathsf{tag}(s), p)$, so $\mathsf{rtag}(v) = \mathsf{tag}(s)$. For the tag-level structured cases (`tuple`, `Promise`), $[\![s]\!]$ is defined as $\{v \mid \mathsf{rtag}(v) = k\}$ where $k = \mathsf{tag}(s)$. For `object` and `algebraic`, Definitions 7 and 8 explicitly require membership in ObjVal (tag 9) and AlgVal (tag 10) respectively, and these tags agree with tag. □

Lemma 6 (**A1** (Tag Disjointness))**.**

$$\forall s_1, s_2 \in \mathsf{SType}.\ \mathsf{tag}(s_1) \neq \mathsf{tag}(s_2) \implies [\![s_1]\!] \cap [\![s_2]\!] = \emptyset$$

Proof. Let $k_1 = \mathsf{tag}(s_1)$ and $k_2 = \mathsf{tag}(s_2)$ with $k_1 \neq k_2$. By Lemma 5, every $v \in [\![s_1]\!]$ satisfies $\mathsf{rtag}(v) = k_1$ and every $v \in [\![s_2]\!]$ satisfies $\mathsf{rtag}(v) = k_2$. No value can have two different tags, so $[\![s_1]\!] \cap [\![s_2]\!] = \emptyset$. □

Definition 7 (Object Values and Object-Type Semantics)**.** *Let* $\mathsf{ObjVal} \subseteq \mathsf{Val}$ *be the set of object-tagged runtime values:*

$$\mathsf{ObjVal} \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 9\}.$$

Assume a (partial) finite-property interface on objects:

$$\mathsf{props} : \mathsf{ObjVal} \to \mathcal{P}_{\mathrm{fin}}(\mathsf{String})$$
$$\mathsf{get} : \mathsf{ObjVal} \times \mathsf{String} \rightharpoonup \mathsf{Val}$$

such that $\mathsf{get}(o, p)$ *is defined exactly when* $p \in \mathsf{props}(o)$*. Write* $\mathsf{has}(o, p)$ *for* $p \in \mathsf{props}(o)$*.*

Object field specifications. *Assume predicates* $\mathsf{callable}, \mathsf{constructible} : \mathsf{ObjVal} \to \{\mathsf{true}, \mathsf{false}\}$ *testing whether an object value is callable or constructible.*

Three-valued signature modes. *To support precise refinement after normalization, we use a three-valued mode for call and constructor signatures:*

$$\mathsf{SigMode} \triangleq \{\mathsf{None},\ \mathsf{Maybe}(\sigma),\ \mathsf{Definite}(\sigma)\}$$

where σ ranges over abstract signatures (we treat signatures opaquely in this document). Intuitively:

- $\mathsf{Definite}(\sigma)$*: the object is definitely callable/constructible with signature σ.*
- $\mathsf{Maybe}(\sigma)$*: the object may or may not be callable/constructible; if it is, it has signature σ.*
- None*: the object is definitely not callable/constructible.*

An object field specification *is a tuple $F = (F_{\text{req}}, F_{\text{opt}}, F_{\text{idx}}, F_{\text{new}}, F_{\text{call}})$ where:*

- $F_{\text{req}} : \mathsf{String} \rightharpoonup \mathsf{TType}$ *is a finite partial map of required fields,*
- $F_{\text{opt}} : \mathsf{String} \rightharpoonup \mathsf{TType}$ *is a finite partial map of optional fields,*
- F_{idx} *is either* None *or* $\mathsf{Some}(K, V)$ *for $K, V \in \mathsf{TType}$ (an index signature),*
- $F_{\text{new}} \in \mathsf{SigMode}$ *for constructor signatures,*
- $F_{\text{call}} \in \mathsf{SigMode}$ *for call signatures.*

Semantics of `object`(F). *We assume the object constructor in* SType *can carry such an F, and define:*

$$o \in [\![\texttt{object}(F)]\!]$$

if and only if $o \in \mathsf{ObjVal}$ and the following constraints hold:

- ***Required fields:*** *for every $p \in \mathrm{dom}(F_{\text{req}})$, $\mathsf{has}(o, p)$ and $\mathsf{get}(o, p) \in [\![F_{\text{req}}(p)]\!]$.*
- ***Optional fields:*** *for every $p \in \mathrm{dom}(F_{\text{opt}})$, if $\mathsf{has}(o, p)$ then $\mathsf{get}(o, p) \in [\![F_{\text{opt}}(p)]\!]$ (and if not, no constraint).*
- ***Index signature (open objects):*** *if $F_{\text{idx}} = \mathsf{Some}(K, V)$ then for every $p \in \mathsf{props}(o)$: if the runtime string value "p" is in $[\![K]\!]$, then $\mathsf{get}(o, p) \in [\![V]\!]$.*
- ***Constructor signature:***
  - *if $F_{\text{new}} = \mathsf{Definite}(\nu)$ then $\mathsf{constructible}(o) = \mathsf{true}$;*
  - *if $F_{\text{new}} = \mathsf{Maybe}(\nu)$ then no constraint (object may or may not be constructible);*
  - *if $F_{\text{new}} = \mathsf{None}$ then $\mathsf{constructible}(o) = \mathsf{false}$.*
- ***Call signature:***
  - *if $F_{\text{call}} = \mathsf{Definite}(\kappa)$ then $\mathsf{callable}(o) = \mathsf{true}$;*
  - *if $F_{\text{call}} = \mathsf{Maybe}(\kappa)$ then no constraint (object may or may not be callable);*
  - *if $F_{\text{call}} = \mathsf{None}$ then $\mathsf{callable}(o) = \mathsf{false}$.*

Note. *This semantics is intentionally* open*: objects may have properties not mentioned in F_{req} or F_{opt}. The three-valued signature modes ensure that normalization (merging callable with non-callable objects) is sound, while still permitting precise refinement via* `typeof`*.*

Definition 8 (Algebraic Values and Tag-Based Algebraic-Type Semantics)**.** *Let* $\mathsf{AlgVal} \subseteq \mathsf{Val}$ *be the set of algebraic-tagged runtime values:*

$$\mathsf{AlgVal} \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 10\}.$$

Recall that $\mathsf{Payload}_{10} = \mathsf{String} \times \mathsf{Payload}'_{10}$ *(Definition 2). Define the runtime discriminant function* $\mathsf{atag} : \mathsf{AlgVal} \to \mathsf{String}$ *by projection:*

$$\mathsf{atag}((10, (\ell, p))) \triangleq \ell.$$

Semantics of `algebraic`(T)**.** *We assume the algebraic constructor in* SType *can carry a finite set of tags* $T \subseteq_{\text{fin}} \mathsf{String}$*, and define:*

$$v \in [\![\texttt{algebraic}(T)]\!] \quad \textit{iff} \quad v \in \mathsf{AlgVal} \ \wedge \ \mathsf{atag}(v) \in T.$$

Note. *This models only the discriminant-level behavior needed for refinement; field/constructor payload constraints are left abstract.*

3 Internal Type Representation

Definition 9 (STypes)**.** STypes *is a finite set* $S \subseteq_{\text{fin}} \mathsf{SType}$ *such that its elements have pairwise distinct runtime tags:*

$$\forall s_1, s_2 \in S.\ s_1 \neq s_2 \implies \mathsf{tag}(s_1) \neq \mathsf{tag}(s_2).$$

Semantics.

$$[\![S]\!] = \bigcup_{s \in S} [\![s]\!].$$

Tags and tag-based lookup. *Define* $\mathsf{tags}(S) \triangleq \{\, \mathsf{tag}(s) \mid s \in S \,\}$*. When* $k \in \mathsf{tags}(S)$*, we write* $S[k]$ *for the unique* $s \in S$ *with* $\mathsf{tag}(s) = k$*.*

Singleton. *We write* $\{s\}$ *for the singleton* STypes *value (a set containing one* SType*).*

Intuition. *An* STypes *value represents the* tag-level *part of a union type: it records, for each runtime tag* k *that may occur, a single representative simple type* s *of that tag.*

Implementation note. *In SKJS, this set is stored and manipulated in a tag-indexed representation for efficiency, but the mathematical model here treats it as a set with the above distinct-tag invariant.*

Example. *The union* `number | string` *is represented as:*

$$S = \{\texttt{number},\ \texttt{string}\}$$

and has semantics $[\![S]\!] = [\![\texttt{number}]\!] \cup [\![\texttt{string}]\!]$*. In contrast,* `number` *and the numeric literal* `1` *share the same tag* 4*, so an* STypes *value cannot simultaneously store both; this is why, for example, a tag-based "diff" cannot precisely express* `number \ 1`*.*

3.1 Normal-Form Operations on STypes

Operations (API). We treat STypes as a normal-form ADT and highlight the operations used by the implementation:

- **Constructors:** $\emptyset \in \mathsf{STypes}$ and $\{s\} \in \mathsf{STypes}$ for $s \in \mathsf{SType}$.
- **Filter-map (refinement primitive):** for tag-preserving g and filter f,

$$\mathsf{filterMap}^{f,g}(S) \triangleq \{\, g(s) \mid s \in S,\ f(s) = \mathsf{true} \,\}.$$

- **Union (implementation union):** $S_1 \sqcup_{\mathsf{STypes}} S_2$ is the normal-form union operation used by SKJS, which merges same-tag alternatives into a single representative per tag (while keeping other tags unchanged).
- **Intersection (over-approximate meet):** the implementation provides a partial operation $S_1 \sqcap_{\mathsf{STypes}} S_2$ (it may return no result when the intersection is bottom). We model it abstractly below via its soundness property.
- **Difference (tag-elimination):** $\mathsf{diff}_{\mathsf{STypes}}(S_1, S_2)$ is defined in Section 47.

Soundness summary. All operations above are intended to be sound with respect to $[\![\cdot]\!]$:

- **Empty/singleton (exact):** $[\![\emptyset]\!] = \emptyset$ and $[\![\{s\}]\!] = [\![s]\!]$.
- **Filter-map (predicate-sound):** if (f, g) is sound for predicate P (Definition 22) and g preserves tags, then

$$[\![S]\!] \cap P \subseteq [\![\mathsf{filterMap}^{f,g}(S)]\!].$$

- **Union (over-approximate union):**

$$[\![S_1]\!] \cup [\![S_2]\!] \subseteq [\![S_1 \sqcup_{\mathsf{STypes}} S_2]\!].$$

- **Intersection (over-approximate intersection):** when $S_1 \sqcap_{\mathsf{STypes}} S_2$ is defined,

$$[\![S_1]\!] \cap [\![S_2]\!] \subseteq [\![S_1 \sqcap_{\mathsf{STypes}} S_2]\!].$$

- **Difference:** (P1) always holds (Lemma 48); (P2) holds under STC (Lemma 50).

Lemma 10 (Union Soundness for STypes)**.** *For all $S_1, S_2 \in \mathsf{STypes}$,*

$$[\![S_1]\!] \cup [\![S_2]\!] \subseteq [\![S_1 \sqcup_{\mathsf{STypes}} S_2]\!].$$

Proof. We will define $S_1 \sqcup_{\mathsf{STypes}} S_2$ to match the implementation's tag-directed union operation (Definition 11), and soundness follows from the corresponding same-tag merge soundness of each case. □

Definition 11 (Implementation Union/Intersection on STypes)**.** *We define $\sqcup_{\mathsf{STypes}}$ and $\sqcap_{\mathsf{STypes}}$ to model SKJS's operations on normal-form tag-distinct unions.*

Same-tag merge on simple types. *Define partial operators* $\sqcup_{\mathsf{SType}}$ *and* $\sqcap_{\mathsf{SType}}$ *on* SType*, used only when* $\mathsf{tag}(s_1) = \mathsf{tag}(s_2)$*. They follow the implementation's tag-dispatch (see* ***`STypes::union`*** *and* ***`STypes::inter`****):*

- ***Primitive units (tags 0/1/2):*** $s_1 \sqcup_{\mathsf{SType}} s_2 \triangleq s_1$ *and* $s_1 \sqcap_{\mathsf{SType}} s_2 \triangleq s_1$*. The implementation keeps the left operand; semantics is unchanged.*
- ***Primitive values (tags 3/4/5/6):*** $\sqcup_{\mathsf{SType}}$ *computes the set union of literal-sets when both are finite, and otherwise yields the full primitive kind (e.g. if either operand is* $\top$*, the result is* $\top$*). For booleans, unioning* $\{\mathsf{true}\}$ *and* $\{\mathsf{false}\}$ *yields* ***`boolean`***$(\top)$*. Intersection* $\sqcap_{\mathsf{SType}}$ *keeps only common literals, or is undefined if none.*
- ***Tuples (tag 7):*** $\sqcup_{\mathsf{SType}}$ *unions element-wise up to the shorter length; missing positions in the shorter tuple are treated as* ***`undefined`*** *and unioned with the corresponding element of the longer tuple.* $\sqcap_{\mathsf{SType}}$ *intersects element-wise up to the shorter length and appends the remainder of the longer tuple unchanged.*
- ***Promises (tag 8):*** $\texttt{Promise}(t_1) \sqcup_{\mathsf{SType}} \texttt{Promise}(t_2) \triangleq \texttt{Promise}(t_1 \sqcup_{\mathsf{TType}} t_2)$*, and similarly* $\sqcap_{\mathsf{SType}}$ *uses* $\sqcap_{\mathsf{TType}}$ *(left abstract here).*
- ***Objects (tag 9):*** *For* $\texttt{object}(F_1) \sqcup_{\mathsf{SType}} \texttt{object}(F_2)$*:*
  - Fields: *union shared field types; make non-shared fields optional.*
  - Index signatures: *union key/value types into a single index signature.*
  - Call/new signatures: *keep only if both sides have one; otherwise drop.*

  Intersection is defined analogously and may be undefined (bottom) in some cases.
- ***Algebraic (tag 10):*** $\texttt{algebraic}(T_1) \sqcup_{\mathsf{SType}} \texttt{algebraic}(T_2)$ *unions the constructor-tag set; when a constructor exists on both sides, it merges its field specifications using object-field union.* $\sqcap_{\mathsf{SType}}$ *intersects constructor-tag sets and merges shared constructors; it is undefined if the tag-set intersection is empty.*

Union on STypes**.** *For* $S_1, S_2 \in \mathsf{STypes}$*, define:*

$$\begin{aligned} S_1 \sqcup_{\mathsf{STypes}} S_2 \triangleq\ & \{\, S_1[k] \mid k \in \mathsf{tags}(S_1) \setminus \mathsf{tags}(S_2) \,\} \\ & \cup \{\, S_2[k] \mid k \in \mathsf{tags}(S_2) \setminus \mathsf{tags}(S_1) \,\} \\ & \cup \{\, S_1[k] \sqcup_{\mathsf{SType}} S_2[k] \mid k \in \mathsf{tags}(S_1) \cap \mathsf{tags}(S_2) \,\}. \end{aligned}$$

Intersection on STypes**.** *Define the (partial) intersection:*

$$\begin{aligned} S_1 \sqcap_{\mathsf{STypes}} S_2 \triangleq \{\, & S_1[k] \sqcap_{\mathsf{SType}} S_2[k] \mid \\ & k \in \mathsf{tags}(S_1) \cap \mathsf{tags}(S_2) \text{ and } S_1[k] \sqcap_{\mathsf{SType}} S_2[k] \text{ defined} \}. \end{aligned}$$

If this set is empty, we treat the result as undefined (bottom), matching the implementation behavior.

Lemma 12 (Soundness of $\sqcup_{\mathsf{STypes}}$ and $\sqcap_{\mathsf{STypes}}$)**.** *For all* $S_1, S_2 \in \mathsf{STypes}$*:*

$$[\![S_1]\!] \cup [\![S_2]\!] \subseteq [\![S_1 \sqcup_{\mathsf{STypes}} S_2]\!].$$

Moreover, when $S_1 \sqcap_{\mathsf{STypes}} S_2$ *is defined,*

$$[\![S_1]\!] \cap [\![S_2]\!] \subseteq [\![S_1 \sqcap_{\mathsf{STypes}} S_2]\!].$$

Proof. For $\sqcup_{\mathsf{STypes}}$: let $v \in [\![S_1]\!] \cup [\![S_2]\!]$. WLOG $v \in [\![S_1]\!]$, so $v \in [\![S_1[k]]\!]$ for some $k \in \mathsf{tags}(S_1)$. If $k \notin \mathsf{tags}(S_2)$, then $S_1[k] \in S_1 \sqcup_{\mathsf{STypes}} S_2$, hence $v \in [\![S_1 \sqcup_{\mathsf{STypes}} S_2]\!]$. If $k \in \mathsf{tags}(S_2)$, then $S_1[k] \sqcup_{\mathsf{STypes}} S_2[k] \in S_1 \sqcup_{\mathsf{STypes}} S_2$, and inclusion reduces to the same-tag merge soundness of $\sqcup_{\mathsf{STypes}}$ for that tag case.

For $\sqcap_{\mathsf{STypes}}$: let $v \in [\![S_1]\!] \cap [\![S_2]\!]$. Then $v \in [\![S_1[k]]\!]$ and $v \in [\![S_2[k]]\!]$ for the same tag k by A1, and inclusion reduces to the same-tag merge soundness of $\sqcap_{\mathsf{STypes}}$ when defined. □

3.2 Completeness and Precision Loss in Normal-Form Operations

We say an operation is *complete* when its result is semantically exact (i.e. no extra values are introduced), and *imprecise* when it introduces a strict over-approximation. This section identifies the specific implementation-driven steps where over-approximation is introduced, to inform potential future precision warnings.

Union is exact for primitive tags. For primitive units and primitive values, $\sqcup_{\mathsf{STypes}}$ is complete: it computes exactly the semantic union of the two alternatives at those tags. (E.g. numeric/string/bigint literal-sets are explicitly stored and unioned; boolean literals union to `boolean`.)

Union is generally imprecise for structured tags. For tuples, objects, promises, and algebraics, the same-tag merge rules may introduce extra inhabitants. Key sources of imprecision:

- **Object field merging via optionalization:** Merging $\texttt{object}(\{a : \tau\})$ and $\texttt{object}(\{b : \upsilon\})$ yields an object type where both a and b are optional. This admits objects containing *both* fields, which need not be in the original disjoint union. This is the primary, intentional precision loss caused by STypes storing at most one object alternative per tag.
- **Index signature union:** When both sides have index signatures, SKJS unions key/value types into a single index signature, losing per-key distinctions (see `TIndexField::union`).
- **Call/new signature merge:** When only one side has a call (or new) signature, SKJS drops it in the merged object. This loses the ability to distinguish callable from non-callable alternatives inside a single object tag.
- **Tuple union padding with `undefined`:** When tuple lengths differ, missing elements in the shorter tuple are unioned with `undefined`, potentially admitting tuples with explicit `undefined` entries not present in the shorter alternative.

Intersection imprecision for structured tags. As with union, $\sqcap_{\mathsf{STypes}}$ is exact for primitive units and primitive values (it intersects literal-sets precisely when they are tracked), but it can introduce over-approximation for structured tags:

- **Index signature intersection uses union (over-approximation):** When intersecting two index signatures, the implementation returns a *single* index signature whose key and value types are *unioned* (see `TIndexField::inter`). This is sound but can admit extra key/value pairs that satisfy neither original signature.
- **Tuple intersection on unequal lengths:** The implementation intersects element-wise up to the shorter length and appends the remaining elements of the longer tuple unchanged. This can admit tuples not in the exact set-theoretic intersection.

Clause-level intersection imprecision. At the clause level (not STypes), `Clauses::inter` accumulates clause-pair intersections in a map keyed by the variable-set. When multiple intersections land on the same key, they are combined using `Clause::union`, widening the result beyond the exact intersection in exchange for keeping at most one clause per key.

Difference is intentionally incomplete at same-tag. $\mathsf{diff}_{\mathsf{STypes}}$ removes *whole tags*. Therefore it cannot express same-tag subtraction such as `number` $\setminus$ `1`, which is why (P2) requires the same-tag coverage side condition STC (Section 49).

Filter-map precision loss. $\mathsf{filterMap}^{f,g}$ can lose precision whenever the filter drops alternatives or whenever the mapper performs a conservative rewrite. In SKJS, this shows up in refinements that (a) retain extra tags conservatively (e.g. `"in"` keeps tuples/promises/algebraics), or (b) perform only tag-level filtering without tracking richer correlations between fields.

Definition 13 (Type Instances)**.** *Let* TInst *be the set of type instances (type variables or generic instantiations).*

Examples. *A type variable instance is something like* T *(a unification variable introduced during type checking; this is* not *a source-level type parameter). A generic instantiation instance is a named type applied to type arguments, e.g.* `Promise`$\langle$`number`$\rangle$ *or* `Array`$\langle$`string`$\rangle$*.*

Each instance $i \in \mathsf{TInst}$ *has a semantic interpretation* $[\![i]\!] \subseteq \mathsf{Val}$*.*

Definition 14 (Type Instance Sets)**.** *Let* $\mathsf{TInsts} = \mathcal{P}_{\mathrm{fin}}(\mathsf{TInst})$ *be the set of* finite *sets of instances. For* $I \in \mathsf{TInsts}$*:*

$$[\![I]\!] = \bigcup_{i \in I} [\![i]\!].$$

Definition 15 (Type Parameters)**.** *Let* TParam *be the set of* type parameters *(generic binders), i.e. names introduced by source-level generic declarations such as* `function f`$\langle T \rangle(\ldots)$ *or* `type Box`$\langle T \rangle = \ldots$*. Type parameters differ from type-variable instances: parameters are binders whose meaning is fixed by the caller, whereas instances are inference unknowns solved by the type checker.*

Definition 16 (Clause)**.** *A clause* $c = (\mathsf{vars}, \mathsf{stypes})$ *consists of:*

- $\mathsf{vars} \subseteq \mathsf{TParam}$*: a finite set of type parameters*
- $\mathsf{stypes} \in \{\mathsf{None}\} \cup \{\mathsf{Some}(S) \mid S \in \mathsf{STypes}\}$

Each type parameter $v \in \mathsf{TParam}$ *has an interpretation* $[\![v]\!] \subseteq \mathsf{Val}$*. Semantics:*

$$[\![(\mathsf{vars}, \mathsf{stypes})]\!] = \left(\bigcap_{v \in \mathsf{vars}} [\![v]\!] \right) \cap [\![\mathsf{stypes}]\!]$$

where $[\![\mathsf{None}]\!] = \mathsf{Val}$ *(top) and* $[\![\mathsf{Some}(S)]\!] = [\![S]\!]$*. For* $\mathsf{vars} = \emptyset$*,* $\bigcap_{v \in \emptyset} [\![v]\!] = \mathsf{Val}$*.*

Well-formedness: $\mathsf{stypes} = \mathsf{None} \implies \mathsf{vars} = \emptyset$ *(i.e.,* $(\emptyset, \mathsf{None})$ *is the unique top clause).*

Definition 17 (Clauses)**.** $\mathsf{Clauses}$ *is a finite map from* $\mathcal{P}_{\mathrm{fin}}(\mathsf{TParam})$ *to* Clause*, with at most one clause per key, and such that keys agree with clause variables: if* $C(k) = (\mathsf{vars}, \mathsf{stypes})$ *then* $\mathsf{vars} = k$*.*

$$[\![C]\!] = \bigcup_{k \in \mathrm{dom}(C)} [\![C(k)]\!]$$

Example. *Let* $S_{\mathsf{num}} = \{\texttt{number}\}$ *and* $S_{\mathsf{str}} = \{\texttt{string}\}$. *Let* $A, B \in \mathsf{TParam}$ *be two type parameters. Consider the clauses map* C *with two entries:*

$$C(\{A\}) = (\{A\}, \mathsf{Some}(S_{\mathsf{num}})) \qquad C(\{B\}) = (\{B\}, \mathsf{Some}(S_{\mathsf{str}})).$$

Then

$$[\![C]\!] = \big([\![A]\!] \cap [\![\texttt{number}]\!]\big) \cup \big([\![B]\!] \cap [\![\texttt{string}]\!]\big).$$

To illustrate a non-singleton key, consider instead the single-entry map C' *defined by:*

$$C'(\{A, B\}) = (\{A, B\}, \mathsf{Some}(S_{\mathsf{num}})).$$

Then $[\![C']\!] = [\![A]\!] \cap [\![B]\!] \cap [\![\texttt{number}]\!]$, *i.e. the clause requires* both *parameters.*

Definition 18 (TType)**.** $\mathsf{TType} = (\mathsf{insts}, \mathsf{clauses})$ *where* $\mathsf{insts} \in \mathsf{TInsts}$ *and* $\mathsf{clauses} \in \mathsf{Clauses}$.

$$[\![(\mathsf{insts}, \mathsf{clauses})]\!] = [\![\mathsf{insts}]\!] \cup [\![\mathsf{clauses}]\!]$$

Schematic. *It is helpful to keep the following picture in mind:*

$$\underbrace{(\mathsf{insts}, \mathsf{clauses})}_{\mathsf{TType}} \quad = \quad \underbrace{\mathsf{insts}}_{\text{“instance” part}} \quad \cup \quad \underbrace{\mathsf{clauses}}_{\text{“conditional” part}}$$

and later $\mathsf{diff}_{\mathsf{TType}}$ *simply applies* diff *componentwise to these two parts.*

Example. *Let* $I = \{\texttt{Promise}\langle\texttt{number}\rangle\} \in \mathsf{TInsts}$ *and let* C *be the clauses map from the previous example. Then* $t = (I, C)$ *denotes the type whose semantics is:*

$$[\![t]\!] = [\![\texttt{Promise}\langle\texttt{number}\rangle]\!] \cup [\![C]\!].$$

4 Subtyping (Skeleton)

We introduce a subtyping judgement on internal types, written $t_1 \leq t_2$ for $t_1, t_2 \in \mathsf{TType}$. In this document, $\leq$ is intended to be a faithful mathematical model of the subtyping *validator* implemented by SKJS (the accept/reject check used by the type checker), rather than an abstract, complete, semantic inclusion relation.

Soundness theorem (to be proved). The central meta-theoretic property we ultimately want from the validator-model judgement $\leq$ is semantic soundness:

$$t_1 \leq t_2 \quad \Longrightarrow \quad [\![t_1]\!] \subseteq [\![t_2]\!].$$

In a full development, this is proved from the subtyping rules/algorithm that define $\leq$.

Incompleteness. The converse implication is intentionally *not* assumed: SKJS may reject some valid semantic inclusions, so $\leq$ can be incomplete.

Note (expansion and substitution effects). In the implementation, the validator operates on expanded/normalized views of types using the current substitution and a variance parameter, and it may update the substitution while checking (as part of inference). We do not model these mechanisms explicitly here; they are absorbed into the meaning of the judgement $t_1 \leq t_2$.

Example obligations (singleton literal types). Once $\leq$ is defined by the validator's rules, it should validate the expected singleton-to-primitive subtypings for primitive literals (these are representative, actually-implemented cases). (In examples we write `1`, `"x"`, `number`, `string` for the corresponding internal types, by abuse of notation.) For example:

$$\texttt{1} \leq \texttt{number} \qquad \texttt{number} \not\leq \texttt{1}.$$

Similarly, a string literal is a subtype of `string`:

$$\texttt{"x"} \leq \texttt{string}.$$

5 Refinement (Predicate-Driven Narrowing)

Definition 19 (Refinement)**.** *Let Γ range over environments mapping local variables to* TType*. Let σ range over runtime environments mapping local variables to runtime values in* Val*. We write $\Gamma(x)$ for lookup and $\Gamma[x \mapsto t]$ for pointwise update.*

Satisfaction. *Define $\sigma \models \Gamma$ to mean: for every local x, we have $\sigma(x) \in [\![\Gamma(x)]\!]$.*

Refinement judgements. *We write:*

$$\Gamma \vdash e \overset{\mathsf{t}}{\rightsquigarrow} \Gamma' \qquad \textit{and} \qquad \Gamma \vdash e \overset{\mathsf{f}}{\rightsquigarrow} \Gamma'$$

to mean "refine under the assumption that e evaluates to `true`/`false`" respectively. These judgements are defined by the inference rules in Section 5.1, and are intentionally partial: they only model the refinement patterns implemented in SKJS.

Soundness. *We call the judgement system* branch-sound *if:*

- *whenever $\sigma \models \Gamma$ and e evaluates to `true` under σ (according to the evaluation relation for refinement tests in Definition 21), then $\sigma \models \Gamma'$ holds for every derivation $\Gamma \vdash e \overset{\mathsf{t}}{\rightsquigarrow} \Gamma'$;*
- *whenever $\sigma \models \Gamma$ and e evaluates to `false` under σ, then $\sigma \models \Gamma'$ holds for every derivation $\Gamma \vdash e \overset{\mathsf{f}}{\rightsquigarrow} \Gamma'$.*

Implementation correspondence. In SKJS, control-flow narrowing is implemented by rewriting the internal type for a local variable using a tag-indexed filter/map on simple types (see the `FilterType` transformer and `setLocal` in the implementation). This motivates defining refinements compositionally by lifting a transformation on SType values to STypes, Clause, Clauses, and finally TType.

Definition 20 (`typeof` Predicates)**.** *Using the* callable *predicate from Definition 7, define the following predicates:*

$$
\begin{aligned}
P_{\mathsf{typeof}=\texttt{"string"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 5\} \\
P_{\mathsf{typeof}=\texttt{"number"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 4\} \\
P_{\mathsf{typeof}=\texttt{"bigint"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 6\} \\
P_{\mathsf{typeof}=\texttt{"boolean"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 3\} \\
P_{\mathsf{typeof}=\texttt{"function"}} &\triangleq \{o \in \mathsf{ObjVal} \mid \mathsf{callable}(o)\} \\
P_{\mathsf{typeof}=\texttt{"undefined"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) \in \{0,1\}\} \\
P_{\mathsf{typeof}=\texttt{"object"}} &\triangleq \mathsf{ObjVal} \cup \mathsf{AlgVal} \cup [\![\texttt{null}]\!].
\end{aligned}
$$

Note. *The* `"symbol"` *case is intentionally omitted because SKJS performs no refinement for it.*

Definition 21 (Evaluation for Refinement Tests)**.** *We define a partial evaluation relation for the refinement-test fragment:*

$$\sigma \models_{\mathsf{true}} e \qquad \textit{and} \qquad \sigma \models_{\mathsf{false}} e.$$

It is defined only for the supported test forms:

- ***Nullish:*** $\sigma \models_{\mathsf{true}} (x \texttt{ === null})$ *iff* $\mathsf{rtag}(\sigma(x)) = 2$*;* $\sigma \models_{\mathsf{false}}$ *is the negation. Similarly for* `undefined`*, with* $\mathsf{rtag}(\sigma(x)) \in \{0,1\}$*.*
- ***typeof:*** $\sigma \models_{\mathsf{true}} (\texttt{typeof } x \texttt{ === } k)$ *iff* $\sigma(x) \in P_{\mathsf{typeof}=k}$ *where* $P_{\mathsf{typeof}=k}$ *is as in Definition 20; the false relation is negation.*
- ***in:*** $\sigma \models_{\mathsf{true}} (\texttt{"}p\texttt{" in } x)$ *iff* $\sigma(x) \in \mathsf{ObjVal}$ *and* $\mathsf{has}(\sigma(x), p)$*; the false relation is negation.*
- ***Algebraic tag equality:*** $\sigma \models_{\mathsf{true}} (x._\texttt{tag} \texttt{ === } \ell)$ *iff* $\sigma(x) \in \mathsf{AlgVal}$ *and* $\mathsf{atag}(\sigma(x)) = \ell$*, where* $\ell \in \mathsf{String}$ *is a string literal; the false relation is negation.*
- ***instanceof:*** $\sigma \models_{\mathsf{true}} (x \texttt{ instanceof } C)$ *iff* $\sigma(x) \in P_{\mathsf{inst}}(C)$ *for a predicate* $P_{\mathsf{inst}}(C) \subseteq \mathsf{Val}$ *(see Definition 42); the false relation is negation.*
- ***Connectives:*** $\sigma \models_{\mathsf{true}} (e_1 \texttt{ \&\& } e_2)$ *iff* $\sigma \models_{\mathsf{true}} e_1$ *and* $\sigma \models_{\mathsf{true}} e_2$*.* $\sigma \models_{\mathsf{false}} (e_1 \texttt{ || } e_2)$ *iff* $\sigma \models_{\mathsf{false}} e_1$ *and* $\sigma \models_{\mathsf{false}} e_2$*. (Other combinations are omitted, since SKJS only uses these connective cases for refinement in the current implementation.)*

Definition 22 (Filter/Map Refinement (Implementation Model))**.** *Let* $P \in \mathcal{P}(\mathsf{Val})$ *be a predicate. Let* $f : \mathsf{SType} \to \{\mathsf{true}, \mathsf{false}\}$ *be a filter and* $g : \mathsf{SType} \to \mathsf{SType}$ *be a mapper. We say* (f, g) *is* sound for P *if for every* $s \in \mathsf{SType}$ *and every* $v \in [\![s]\!]$*:*

$$v \in P \implies (f(s) = \mathsf{true} \ \wedge \ v \in [\![g(s)]\!]).$$

Remark (tag preservation). *In the implementation, the mapper* g *used for refinement preserves runtime tags:* $\mathsf{tag}(g(s)) = \mathsf{tag}(s)$*. In this document we assume tag preservation when defining* $\mathsf{refine}^{f,g}_{\mathsf{STypes}}$*, so that the result is again a well-formed* STypes *set (pairwise distinct tags). This matches SKJS's* `FilterType` *implementation.*

Lift to STypes. *Define:*

$$\mathsf{refine}^{f,g}_{\mathsf{STypes}}(S) \triangleq \{\, g(s) \mid s \in S,\ f(s) = \mathsf{true} \,\}.$$

Operationally, this is exactly the "filter-map" over the unique representative per tag.

Lift to Clause. *For a clause* $c = (\mathsf{vars}, \mathsf{stypes})$ *define:*

$$\mathsf{refine}^{f,g}_{\mathsf{Clause}}(c) \triangleq \begin{cases} (\mathsf{vars}, \mathsf{None}) & \text{if } \mathsf{stypes} = \mathsf{None} \\ (\mathsf{vars}, \mathsf{Some}(\mathsf{refine}^{f,g}_{\mathsf{STypes}}(S))) & \text{if } \mathsf{stypes} = \mathsf{Some}(S) \end{cases}$$

and interpret the special case $\mathsf{Some}(\emptyset)$ *as* $\bot$ *(dropped), matching the implementation invariant that bottom clauses are not stored.*

Lift to Clauses. *For* $C \in \mathsf{Clauses}$*, define* $\mathsf{refine}^{f,g}_{\mathsf{Clauses}}(C)$ *by applying* $\mathsf{refine}^{f,g}_{\mathsf{Clause}}$ *to every clause and dropping* $\bot$ *results.*

Lift to TType. *For* $t = (\mathsf{insts}, \mathsf{clauses})$ *define:*

$$\mathsf{refine}^{f,g}(t) \triangleq (\mathsf{insts}, \mathsf{refine}^{f,g}_{\mathsf{Clauses}}(\mathsf{clauses})).$$

This matches the current implementation: refinement rewrites the clause part but does not filter the instance part.

Definition 23 (Defaulting Refinement (Model of `setLocal`))**.** *Some refinements in SKJS provide a* default *type when the current type is too imprecise to refine (notably when it is top or has no informative alternatives). This is modeled by a* defaulting refinement *operator.*

Let $R : \mathsf{TType} \to \mathsf{TType}$ *be a transformer and let* d *be an* optional *default simple type (*$d \in \{\mathsf{None}\} \cup \{\mathsf{Some}(s) \mid s \in \mathsf{SType}\}$*).*

Singleton constructor. *Write* $\mathsf{single}(s)$ *for the* TType *value consisting of a single clause with* $\mathsf{STypes} = \{s\}$ *and no instances.*

Top/empty test. *We say a* TType *value is* syntactically top *if its clause component is the unique top clause* $(\emptyset, \mathsf{None})$*, and* syntactically empty *if its clause component is empty (bottom).*

Definition. *Define* $\mathsf{def}(R, d) : \mathsf{TType} \to \mathsf{TType}$ *by:*

$$\mathsf{def}(R, d)(t) \triangleq \begin{cases} R(t) & \text{if } d = \mathsf{None} \\ \mathsf{single}(s) & \text{if } d = \mathsf{Some}(s) \text{ and } R(t) \text{ is syntactically top or empty} \\ R(t) & \text{otherwise} \end{cases}$$

Note. *This abstracts the implementation check in* `setLocal` *based on* `getSTypes` *(after expansion): it injects a default when refinement would otherwise leave a top/empty shape.*

Definition 24 (Predicate-Sound Type Transformer)**.** *Let* $P \in \mathcal{P}(\mathsf{Val})$ *and let* $R : \mathsf{TType} \to \mathsf{TType}$*. We say* R *is* predicate-sound for P *if for all* $t \in \mathsf{TType}$*:*

$$[\![t]\!] \cap P \subseteq [\![R(t)]\!].$$

Lemma 25 (Filter/Map Refinement is Predicate-Sound)**.** *If (f, g) is sound for P, then the transformer $t \mapsto \mathsf{refine}^{f,g}(t)$ is predicate-sound for P (Definition 24).*

Proof. Let $t = (\mathsf{insts}, \mathsf{clauses})$. We prove $[\![t]\!] \cap P \subseteq [\![\mathsf{refine}^{f,g}(t)]\!]$ by cases on membership in $[\![\mathsf{insts}]\!] \cup [\![\mathsf{clauses}]\!]$.

- If $v \in [\![\mathsf{insts}]\!] \cap P$, then $v \in [\![\mathsf{insts}]\!] \subseteq [\![\mathsf{refine}^{f,g}(t)]\!]$ since refinement leaves insts unchanged.
- If $v \in [\![\mathsf{clauses}]\!] \cap P$, then $v \in [\![(\mathsf{vars}, \mathsf{stypes})]\!]$ for some clause $(\mathsf{vars}, \mathsf{stypes})$ in clauses. By clause semantics (Definition 16), $v \in \bigcap_{w \in \mathsf{vars}} [\![w]\!]$ and $v \in [\![\mathsf{stypes}]\!]$. If $\mathsf{stypes} = \mathsf{None}$, then $\mathsf{refine}^{f,g}_{\mathsf{Clause}}(\mathsf{vars}, \mathsf{None}) = (\mathsf{vars}, \mathsf{None})$ and the clause remains top, so v is still included. If $\mathsf{stypes} = \mathsf{Some}(S)$, then $v \in [\![S]\!] = \bigcup_{s \in S} [\![s]\!]$, so $v \in [\![s]\!]$ for some $s \in S$. By soundness of (f, g) for P and $v \in P$, we have $f(s) = \mathsf{true}$ and $v \in [\![g(s)]\!]$. Hence $v \in [\![\mathsf{refine}^{f,g}_{\mathsf{STypes}}(S)]\!]$. The refined clause is $(\mathsf{vars}, \mathsf{Some}(\mathsf{refine}^{f,g}_{\mathsf{STypes}}(S)))$, and since vars is unchanged and $v \in \bigcap_{w \in \mathsf{vars}} [\![w]\!]$, we have $v \in [\![(\mathsf{vars}, \mathsf{Some}(\mathsf{refine}^{f,g}_{\mathsf{STypes}}(S)))]\!]$. Therefore $v \in [\![\mathsf{refine}^{f,g}_{\mathsf{Clauses}}(\mathsf{clauses})]\!]$.

Combining cases yields the desired inclusion. □

Lemma 26 (Defaulting Preserves Predicate-Soundness)**.** *Let $R : \mathsf{TType} \to \mathsf{TType}$ be predicate-sound for $P \subseteq \mathsf{Val}$. Let d be an optional default. If $d = \mathsf{None}$, then $\mathsf{def}(R, d)$ is predicate-sound for P. If $d = \mathsf{Some}(s)$ and $P \subseteq [\![\mathsf{single}(s)]\!]$, then $\mathsf{def}(R, d)$ is predicate-sound for P.*

Proof. If $d = \mathsf{None}$, then $\mathsf{def}(R, d) = R$ by Definition 23, so predicate-soundness is immediate. If $d = \mathsf{Some}(s)$, let $t \in \mathsf{TType}$ and let $v \in [\![t]\!] \cap P$. If $R(t)$ is not syntactically top/empty, then $\mathsf{def}(R, d)(t) = R(t)$ and $v \in [\![R(t)]\!]$ by predicate-soundness of R. If $R(t)$ is syntactically top/empty, then $\mathsf{def}(R, d)(t) = \mathsf{single}(s)$, and by hypothesis $v \in P \subseteq [\![\mathsf{single}(s)]\!]$. Thus $[\![t]\!] \cap P \subseteq [\![\mathsf{def}(R, d)(t)]\!]$ for all t. □

5.1 Algorithmic Refinement Judgements (Inference-Rule Style)

Local update helper. Given a local x and a refinement transformer $R : \mathsf{TType} \to \mathsf{TType}$, define:

$$\mathsf{upd}(\Gamma, x, R) \triangleq \Gamma[x \mapsto R(\Gamma(x))].$$

When a rule refines a non-local expression, it produces Γ.

Default behavior. If no rule applies to e (i.e. e is not one of the supported refinement-test forms), then refinement leaves the environment unchanged.

Strict inequality. The implementation treats strict inequality `!==` by flipping the target (i.e. refining for `false` of the corresponding strict equality). Accordingly, the refinement rules for $(x \mathrel{!}== y)$ are the same as for $(x === y)$, with $\Rightarrow$ and $\Leftarrow$ swapped.

Nullish filters and mappers.

Definition 27 (Nullish Filters and Mappers)**.** *Define the following filter/map functions on simple types:*

$$g_{\mathrm{id}}(s) \triangleq s.$$

$$f_{\mathsf{null}}(s) \triangleq (\mathsf{tag}(s) = 2), \qquad f_{\neg\mathsf{null}}(s) \triangleq (\mathsf{tag}(s) \neq 2).$$

$$f_{\mathsf{undef}}(s) \triangleq (\mathsf{tag}(s) \in \{0, 1\}), \qquad f_{\neg\mathsf{undef}}(s) \triangleq (\mathsf{tag}(s) \notin \{0, 1\}).$$

Nullish equality.

$$\frac{}{\Gamma \vdash (x \texttt{ === null}) \overset{\mathsf{t}}{\rightsquigarrow} \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{refine}^{f_{\mathsf{null}}, g_{\mathrm{id}}}(t))}\ \text{R-NullT}$$

$$\frac{}{\Gamma \vdash (x \texttt{ === null}) \overset{\mathsf{f}}{\rightsquigarrow} \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{refine}^{f_{\neg\mathsf{null}}, g_{\mathrm{id}}}(t))}\ \text{R-NullF}$$

$$\frac{}{\Gamma \vdash (x \texttt{ === undefined}) \overset{\mathsf{t}}{\rightsquigarrow} \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{refine}^{f_{\mathsf{undef}}, g_{\mathrm{id}}}(t))}\ \text{R-UndefT}$$

$$\frac{}{\Gamma \vdash (x \texttt{ === undefined}) \overset{\mathsf{f}}{\rightsquigarrow} \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{refine}^{f_{\neg\mathsf{undef}}, g_{\mathrm{id}}}(t))}\ \text{R-UndefF}$$

where $f_{\mathsf{null}}, f_{\neg\mathsf{null}}, f_{\mathsf{undef}}, f_{\neg\mathsf{undef}}, g_{\mathrm{id}}$ are as in Definition 27.

Side condition (locals only). In the implementation, comparisons against `null`/`undefined` are only permitted when the compared expression is a local variable; otherwise a typing error is raised. Accordingly, the rules above are intended only for tests where x is a local.

`typeof` filters and defaults.

Definition 28 (`typeof` Filters and Defaults)**.** *For each k in Definition 20, we define a filter/map pair and an optional default d_k matching the implementation.*

- *For $k \in \{$`"string"`, `"number"`, `"bigint"`, `"boolean"`, `"undefined"`$\}$: $f_k(s)$ is true iff* $\mathsf{tag}(s)$ *is the corresponding tag, and $g_k \triangleq g_{\mathrm{id}}$.*
- *For $k =$ `"object"`: $f_{\texttt{"object"}}(s)$ is true iff* $\mathsf{tag}(s) \in \{2, 9, 10\}$ *(object, null, or algebraic), and $g_{\texttt{"object"}} \triangleq g_{\mathrm{id}}$.*
- *For $k =$ `"function"`:*
  - *$f_{\texttt{"function"}}(s) =$* true *iff $s =$* $\mathtt{object}(F)$ *with $F_{\mathrm{call}} \in \{\mathsf{Definite}(\kappa), \mathsf{Maybe}(\kappa)\}$.*
  - *$g_{\texttt{"function"}}(s)$: if $F_{\mathrm{call}} = \mathsf{Maybe}(\kappa)$, return* $\mathtt{object}(F')$ *with $F'_{\mathrm{call}} = \mathsf{Definite}(\kappa)$; otherwise return s.*

False-branch filter and mapper for `"function"`. *For the false branch of* `typeof x === "function"`*, we use:*

- *$f_{\neg\texttt{"function"}}(s) =$* true *iff s is not an object, or $s =$* $\mathtt{object}(F)$ *with $F_{\mathrm{call}} \in \{\mathsf{None}, \mathsf{Maybe}(\kappa)\}$.*
- *$g_{\neg\texttt{"function"}}(s)$: if $F_{\mathrm{call}} = \mathsf{Maybe}(\kappa)$, set $F'_{\mathrm{call}} = \mathsf{None}$; else return s.*

Note. *The key insight is that* $\mathsf{Maybe}(\kappa)$ *types are kept in* both *branches: the true branch refines them to* $\mathsf{Definite}(\kappa)$*, and the false branch refines them to* None*. This ensures predicate-soundness for both branches after normalization. Define defaults:*

- *$d_{\texttt{"string"}} =$* $\mathsf{Some}(\mathtt{string})$*, $d_{\texttt{"number"}} =$* $\mathsf{Some}(\mathtt{number})$*, $d_{\texttt{"bigint"}} =$* $\mathsf{Some}(\mathtt{bigint})$*, $d_{\texttt{"boolean"}} =$* $\mathsf{Some}(\mathtt{boolean})$*.*
- *Let $F_\emptyset$ be the empty object field specification (no required/optional fields, no index signature, no* `new`/`call` *signature), and set $d_{\texttt{"object"}} =$* $\mathsf{Some}(\mathtt{object}(F_\emptyset))$*.*
- *$d_{\texttt{"function"}} =$* None *and $d_{\texttt{"undefined"}} =$* None*.*

Symbol. *For $k =$ `"symbol"`, SKJS performs no refinement, so we omit f_k, g_k, d_k.*

`typeof`.

$$\frac{}{\Gamma \vdash (\texttt{typeof}\ x === k) \overset{\mathsf{t}}{\rightsquigarrow} \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{def}(t' \mapsto \mathsf{refine}^{f_k, g_k}(t'),\ d_k)(t))}\ \text{R-TypeofT}$$

$$\frac{}{\Gamma \vdash (\texttt{typeof}\ x === k) \overset{\mathsf{f}}{\rightsquigarrow} \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{refine}^{f_k^{\neg}, g_k^{\neg}}(t))}\ \text{R-TypeofF}$$

where f_k, g_k, d_k are as in Definition 28, and $(f_k^{\neg}, g_k^{\neg}) = (\neg f_k, g_{\mathrm{id}})$ for most k, except for $k =$ `"function"` where $(f^{\neg}_{\texttt{"function"}}, g^{\neg}_{\texttt{"function"}}) = (f_{\neg\texttt{"function"}}, g_{\neg\texttt{"function"}})$ as defined in Definition 28.

`typeof "symbol"`. The implementation performs no refinement for `typeof x === "symbol"`; both branches leave the environment unchanged.

`in`.

$$\frac{}{\Gamma \vdash (\texttt{"}p\texttt{"}\ \texttt{in}\ x) \overset{\mathsf{t}}{\rightsquigarrow} \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{def}(t' \mapsto \mathsf{refine}^{f_p^{\mathsf{true}}, g_p^{\mathsf{true}}}(t'),\ d_p^{\mathsf{in}})(t))}\ \text{R-InT}$$

$$\frac{}{\Gamma \vdash (\texttt{"}p\texttt{"}\ \texttt{in}\ x) \overset{\mathsf{f}}{\rightsquigarrow} \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{refine}^{f_p^{\mathsf{false}}, g_p^{\mathsf{false}}}(t))}\ \text{R-InF}$$

where $d_p^{\mathsf{in}} = \mathsf{Some}(\texttt{object}(F_p))$ is a default object type requiring property p with top value type (Definition 7). This corresponds to the implementation's behavior when the left-hand side is a single string literal.

Algebraic tag equality.

$$\frac{}{\Gamma \vdash (x.\texttt{_tag} === \ell) \overset{\mathsf{t}}{\rightsquigarrow} \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{refine}^{f_\ell^{\mathsf{alg}}, g_\ell^{\mathsf{alg,true}}}(t))}\ \text{R-AlgTagT}$$

$$\frac{}{\Gamma \vdash (x.\texttt{_tag} === \ell) \overset{\mathsf{f}}{\rightsquigarrow} \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{refine}^{f_\ell^{\mathsf{alg}}, g_\ell^{\mathsf{alg,false}}}(t))}\ \text{R-AlgTagF}$$

where $\ell \in \mathsf{String}$ is a string literal.

`instanceof`.

$$\frac{}{\Gamma \vdash (x\ \texttt{instanceof}\ C) \overset{\mathsf{t}}{\rightsquigarrow} \mathsf{upd}(\Gamma, x,\ t \mapsto T_{\mathsf{inst}}(C))}\ \text{R-InstT}$$

$$\frac{}{\Gamma \vdash (x\ \texttt{instanceof}\ C) \overset{\mathsf{f}}{\rightsquigarrow} \Gamma}\ \text{R-InstF}$$

where $T_{\mathsf{inst}}(C)$ is the instance type produced by the implementation (Definition 42).

$$\frac{}{\Gamma \vdash (x\ \texttt{instanceof Array}) \overset{\mathsf{t}}{\rightsquigarrow} \mathsf{upd}(\Gamma, x,\ t \mapsto T_{\mathsf{array}})}\ \text{R-InstArrayT}$$

where T_{array} is the implementation's chosen array instance type (e.g. $\texttt{Array}\langle\top\rangle$).

Boolean connectives. The implementation composes refinements sequentially. For example:

$$\frac{\Gamma \vdash e_1 \overset{\mathsf{t}}{\rightsquigarrow} \Gamma_1 \qquad \Gamma_1 \vdash e_2 \overset{\mathsf{t}}{\rightsquigarrow} \Gamma_2}{\Gamma \vdash (e_1 \mathbin{\&\&} e_2) \overset{\mathsf{t}}{\rightsquigarrow} \Gamma_2}\ \text{R-AndT} \qquad \frac{\Gamma \vdash e_1 \overset{\mathsf{f}}{\rightsquigarrow} \Gamma_1 \qquad \Gamma_1 \vdash e_2 \overset{\mathsf{f}}{\rightsquigarrow} \Gamma_2}{\Gamma \vdash (e_1 \mathbin{||} e_2) \overset{\mathsf{f}}{\rightsquigarrow} \Gamma_2}\ \text{R-OrF}$$

Lemma 29 (Predicate-Sound Transformers Yield Branch-Sound Rules)**.** *Let $R : \mathsf{TType} \to \mathsf{TType}$ be predicate-sound for predicate $P \subseteq \mathsf{Val}$. Suppose the inference rule for expression e updates the environment via $\mathsf{upd}(\Gamma, x, R)$. If $\sigma \models_{\mathsf{true}} e$ implies $\sigma(x) \in P$, then the rule is branch-sound for the true branch. Analogously for the false branch with $\sigma \models_{\mathsf{false}} e$.*

Proof. Suppose $\sigma \models \Gamma$ and $\sigma \models_{\mathsf{true}} e$. Let $\Gamma' = \mathsf{upd}(\Gamma, x, R)$. For any local $y \neq x$, $\Gamma'(y) = \Gamma(y)$ and $\sigma(y) \in [\![\Gamma(y)]\!] = [\![\Gamma'(y)]\!]$. For x: by hypothesis $\sigma(x) \in P$, and by $\sigma \models \Gamma$ we have $\sigma(x) \in [\![\Gamma(x)]\!]$. Thus $\sigma(x) \in [\![\Gamma(x)]\!] \cap P \subseteq [\![R(\Gamma(x))]\!] = [\![\Gamma'(x)]\!]$ by predicate-soundness of R. Hence $\sigma \models \Gamma'$. □

Lemma 30 (Connective Rules are Branch-Sound)**.** *Assume that for every derivation $\Gamma \vdash e_1 \overset{\mathsf{t}}{\rightsquigarrow} \Gamma_1$, the rule is branch-sound for the true branch in the sense of Definition 19. Assume likewise that for every derivation $\Gamma_1 \vdash e_2 \overset{\mathsf{t}}{\rightsquigarrow} \Gamma_2$, the rule is branch-sound for the true branch. Then the connective rule **R-AndT** is branch-sound for the true branch.*

*Dually, assume that for every derivation $\Gamma \vdash e_1 \overset{\mathsf{f}}{\rightsquigarrow} \Gamma_1$ and $\Gamma_1 \vdash e_2 \overset{\mathsf{f}}{\rightsquigarrow} \Gamma_2$, the rules are branch-sound for the false branch. Then the connective rule **R-OrF** is branch-sound for the false branch.*

Proof. **R-AndT.** Suppose $\sigma \models \Gamma$ and $\sigma \models_{\mathsf{true}} (e_1 \mathbin{\&\&} e_2)$. By Definition 21, we have $\sigma \models_{\mathsf{true}} e_1$ and $\sigma \models_{\mathsf{true}} e_2$. Consider a derivation of $\Gamma \vdash (e_1 \mathbin{\&\&} e_2) \overset{\mathsf{t}}{\rightsquigarrow} \Gamma_2$ via **R-AndT**; it contains premises $\Gamma \vdash e_1 \overset{\mathsf{t}}{\rightsquigarrow} \Gamma_1$ and $\Gamma_1 \vdash e_2 \overset{\mathsf{t}}{\rightsquigarrow} \Gamma_2$. By branch-soundness of the first premise and $\sigma \models_{\mathsf{true}} e_1$, from $\sigma \models \Gamma$ we obtain $\sigma \models \Gamma_1$. Then by branch-soundness of the second premise and $\sigma \models_{\mathsf{true}} e_2$, we obtain $\sigma \models \Gamma_2$. This is exactly branch-soundness of **R-AndT**.

R-OrF. Suppose $\sigma \models \Gamma$ and $\sigma \models_{\mathsf{false}} (e_1 \mathbin{||} e_2)$. By Definition 21, we have $\sigma \models_{\mathsf{false}} e_1$ and $\sigma \models_{\mathsf{false}} e_2$. For a derivation of $\Gamma \vdash (e_1 \mathbin{||} e_2) \overset{\mathsf{f}}{\rightsquigarrow} \Gamma_2$ via **R-OrF**, we similarly obtain $\sigma \models \Gamma_1$ from the first premise and then $\sigma \models \Gamma_2$ from the second premise. □

Soundness connection. Each of the case-study lemmas in the following subsections establishes that the filter/map pair (f, g) is sound for the corresponding runtime predicate P, which by Lemma 25 implies predicate-soundness of $\mathsf{refine}^{f,g}$, and by Lemma 29 implies branch-soundness of the inference rule.

5.2 Refinement Case Study: the `in` Operator

We formalize the refinement performed for conditions of the form `"p" in x` in SKJS. The implementation refines only local variables and updates object types by either (a) adding a required field on the true branch, or (b) forgetting a field on the false branch.

Definition 31 (`in` Predicates)**.** *Fix a property name $p \in \mathsf{String}$. Define:*

$$P_{\text{in}}^{\mathsf{true}}(p) \triangleq \{\, o \in \mathsf{Val} \mid o \in \mathsf{ObjVal} \wedge \mathsf{has}(o, p) \,\} \qquad P_{\text{in}}^{\mathsf{false}}(p) \triangleq \mathsf{Val} \setminus P_{\text{in}}^{\mathsf{true}}(p).$$

Definition 32 (`in` Filter/Map Pair)**.** *Fix $p \in \mathsf{String}$ and let $\top_{\mathsf{TType}}$ be any type with $[\![\top_{\mathsf{TType}}]\!] = \mathsf{Val}$ (e.g. the unique top clause). Let F_p be the object field specification with F_{req} mapping $p \mapsto \top_{\mathsf{TType}}$, F_{opt} empty, $F_{\text{idx}} = \mathsf{None}$, and $F_{\text{new}} = F_{\text{call}} = \mathsf{None}$. Define a mapper $g_p^{\mathsf{true}} : \mathsf{SType} \to \mathsf{SType}$ as follows:*

- *If s is an object type $s = \texttt{object}(F)$ and $F_{\text{idx}} = \mathsf{None}$ and $p \notin \text{dom}(F_{\text{req}})$, then $g_p^{\mathsf{true}}(s)$ is $\texttt{object}(F')$ where F' is F except that F'_{req} additionally maps $p \mapsto \top_{\mathsf{TType}}$.*
- *Otherwise, $g_p^{\mathsf{true}}(s) = s$.*

Let $f_p^{\mathsf{true}} : \mathsf{SType} \to \{\mathsf{true}, \mathsf{false}\}$ be the filter defined by:

$$f_p^{\mathsf{true}}(s) = \begin{cases} \mathsf{true} & \text{if } \mathsf{tag}(s) \in \{7, 8, 9, 10\} \text{ (tuple, promise, object, or algebraic)} \\ \mathsf{false} & \text{otherwise (primitives)} \end{cases}$$

Note. *The implementation conservatively retains tuples, promises, and algebraic types (tags 7, 8, 10) in addition to objects; these cases are marked `TODO` in the implementation. Only object-tagged values (tag 9) can actually satisfy the `in` predicate at runtime, so the filter is sound but imprecise for tags 7, 8, and 10.*

Define a mapper $g_p^{\mathsf{false}} : \mathsf{SType} \to \mathsf{SType}$ by:

- *If $s = \texttt{object}(F)$ then $g_p^{\mathsf{false}}(s) = \texttt{object}(F \setminus p)$ where $F \setminus p$ removes p from both F_{req} and F_{opt} (and leaves $F_{\text{idx}}, F_{\text{new}}, F_{\text{call}}$ unchanged).*
- *Otherwise $g_p^{\mathsf{false}}(s) = s$.*

Let $f_p^{\mathsf{false}}(s) = \mathsf{true}$ for all s (the implementation does not attempt to exclude non-objects in the false branch).

Default. *Define $d_p^{\mathsf{in}} \triangleq \mathsf{Some}(\texttt{object}(F_p))$.*

Lemma 33 (`in`-True Branch (with Default) is Predicate-Sound)**.** *For every $p \in \mathsf{String}$, the transformer $t \mapsto \mathsf{def}(t' \mapsto \mathsf{refine}^{f_p^{\mathsf{true}}, g_p^{\mathsf{true}}}(t'), d_p^{\mathsf{in}})(t)$ is predicate-sound for $P_{\text{in}}^{\mathsf{true}}(p)$.*

Proof. Let $R_p(t) \triangleq \mathsf{refine}^{f_p^{\mathsf{true}}, g_p^{\mathsf{true}}}(t)$. We first show that $(f_p^{\mathsf{true}}, g_p^{\mathsf{true}})$ is sound for $P_{\text{in}}^{\mathsf{true}}(p)$ (Definition 22), and conclude that R_p is predicate-sound for $P_{\text{in}}^{\mathsf{true}}(p)$ by Lemma 25. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\text{in}}^{\mathsf{true}}(p)$. Then $v \in \mathsf{ObjVal}$, so $\mathsf{rtag}(v) = 9$. By A0, $\mathsf{rtag}(v) = \mathsf{tag}(s)$, hence $\mathsf{tag}(s) = 9$ and therefore s must be an object type of the form $\texttt{object}(F)$. By definition of $P_{\text{in}}^{\mathsf{true}}(p)$, $\mathsf{has}(v, p)$. The filter keeps object types, so $f_p^{\mathsf{true}}(s) = \mathsf{true}$. If g_p^{true} adds the required field $p \mapsto \top_{\mathsf{TType}}$, then v satisfies the added requirement because it already has p and $\mathsf{get}(v, p) \in [\![\top_{\mathsf{TType}}]\!] = \mathsf{Val}$. Otherwise $g_p^{\mathsf{true}}(s) = s$ and membership is immediate. Thus $v \in [\![g_p^{\mathsf{true}}(s)]\!]$ and soundness holds.

Finally, by Definition 7, every $v \in P_{\text{in}}^{\mathsf{true}}(p)$ is in $[\![\texttt{object}(F_p)]\!] = [\![\mathsf{single}(\texttt{object}(F_p))]\!]$, so $P_{\text{in}}^{\mathsf{true}}(p) \subseteq [\![\mathsf{single}(\texttt{object}(F_p))]\!]$. By Lemma 26, $\mathsf{def}(R_p, d_p^{\mathsf{in}})$ is predicate-sound for $P_{\text{in}}^{\mathsf{true}}(p)$. □

Lemma 34 (`in`-False Branch is Predicate-Sound)**.** *For every $p \in \mathsf{String}$, the refinement $\mathsf{refine}^{f_p^{\mathsf{false}}, g_p^{\mathsf{false}}}$ is predicate-sound for $P_{\text{in}}^{\mathsf{false}}(p)$.*

Proof. We show $(f_p^{\mathsf{false}}, g_p^{\mathsf{false}})$ is sound for $P_{\text{in}}^{\mathsf{false}}(p)$ (Definition 22), and conclude by Lemma 25. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\text{in}}^{\mathsf{false}}(p)$. We have $f_p^{\mathsf{false}}(s) = \mathsf{true}$ by definition. If s is not an object type, then $g_p^{\mathsf{false}}(s) = s$ so $v \in [\![g_p^{\mathsf{false}}(s)]\!]$. If $s = \texttt{object}(F)$, then $g_p^{\mathsf{false}}(s) = \texttt{object}(F \setminus p)$ which removes p from field constraints. By Definition 7, the semantics only constrains listed fields, so removing a field enlarges (or preserves) the inhabitant set: $[\![\texttt{object}(F)]\!] \subseteq [\![\texttt{object}(F \setminus p)]\!]$. Hence $v \in [\![g_p^{\mathsf{false}}(s)]\!]$. □

5.3 Refinement Case Study: Nullish Checks

We model the refinements performed for comparisons of a local variable against `null` and `undefined`. In the implementation, `undefined` and `void` are treated equivalently for refinement.

Definition 35 (Nullish Predicates)**.** *Define the following predicates on runtime values:*

$$P_{\mathsf{null}} \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 2\} \qquad P_{\mathsf{undef}} \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) \in \{0,1\}\}.$$

We also write $\neg P$ *for* $\mathsf{Val} \setminus P$*.*

Lemma 36 (Null Refinement is Predicate-Sound)**.** *Let* f_{null} *and* g_{id} *be as in Definition 27. Then* $\mathsf{refine}^{f_{\mathsf{null}}, g_{\mathrm{id}}}$ *is predicate-sound for* P_{null}*. Moreover, letting* $f_{\neg\mathsf{null}}$ *be as in Definition 27,* $\mathsf{refine}^{f_{\neg\mathsf{null}}, g_{\mathrm{id}}}$ *is predicate-sound for* $\neg P_{\mathsf{null}}$*.*

Proof. We show $(f_{\mathsf{null}}, g_{\mathrm{id}})$ is sound for P_{null} (Definition 22) and apply Lemma 25. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\mathsf{null}}$. Then $\mathsf{rtag}(v) = 2$, and by A0 we have $\mathsf{tag}(s) = 2$, hence $f_{\mathsf{null}}(s) = \mathsf{true}$. Since g_{id} is the identity, $v \in [\![g_{\mathrm{id}}(s)]\!]$ is immediate. The $\neg P_{\mathsf{null}}$ case is analogous: if $v \in \neg P_{\mathsf{null}}$ then $\mathsf{rtag}(v) \neq 2$, so by A0 $\mathsf{tag}(s) \neq 2$ and $f_{\neg\mathsf{null}}(s) = \mathsf{true}$. □

Lemma 37 (Undefined/Void Refinement is Predicate-Sound)**.** *Let* f_{undef} *and* g_{id} *be as in Definition 27. Then* $\mathsf{refine}^{f_{\mathsf{undef}}, g_{\mathrm{id}}}$ *is predicate-sound for* P_{undef}*. Moreover, letting* $f_{\neg\mathsf{undef}}$ *be as in Definition 27,* $\mathsf{refine}^{f_{\neg\mathsf{undef}}, g_{\mathrm{id}}}$ *is predicate-sound for* $\neg P_{\mathsf{undef}}$*.*

Proof. We show $(f_{\mathsf{undef}}, g_{\mathrm{id}})$ is sound for P_{undef} (Definition 22) and apply Lemma 25. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\mathsf{undef}}$, i.e., $\mathsf{rtag}(v) \in \{0,1\}$. By A0, $\mathsf{tag}(s) = \mathsf{rtag}(v) \in \{0,1\}$, so $f_{\mathsf{undef}}(s) = \mathsf{true}$ and $v \in [\![g_{\mathrm{id}}(s)]\!]$. The $\neg P_{\mathsf{undef}}$ case is analogous. □

5.4 Refinement Case Study: `typeof` Checks

We model the refinements performed for tests of the form `typeof x === "k"` for a subset of strings k. The implementation refines based on the internal tag/classification of the current SType alternatives.

Note. The predicate modeled here is the one used by the checker for narrowing; it may be a conservative approximation of JavaScript runtime `typeof`. The predicates and the corresponding filter/default choices are defined in Definitions 20 and 28.

Lemma 38 (`typeof` Refinement is Predicate-Sound)**.** *For each* k *in Definition 20, the transformer* $t \mapsto \mathsf{refine}^{f_k, g_k}(t)$ *is predicate-sound for* $P_{\mathsf{typeof}=k}$*, and the false-branch transformer* $t \mapsto \mathsf{refine}^{f_k^\neg, g_k^\neg}(t)$ *is predicate-sound for* $\neg P_{\mathsf{typeof}=k}$*, where* $(f_k^\neg, g_k^\neg) = (\neg f_k, g_{\mathrm{id}})$ *for most* k*, except for* $k =$ *"function" where* $(f^\neg_{\text{"function"}}, g^\neg_{\text{"function"}}) = (f_{\neg\text{"function"}}, g_{\neg\text{"function"}})$ *as in Definition 28.*

Defaults. *If* $d_k = \mathsf{Some}(s_k)$ *satisfies* $P_{\mathsf{typeof}=k} \subseteq [\![\mathsf{single}(s_k)]\!]$*, then the defaulted transformer*

$$t \mapsto \mathsf{def}(t' \mapsto \mathsf{refine}^{f_k, g_k}(t'), d_k)(t)$$

is predicate-sound for $P_{\mathsf{typeof}=k}$*.*

Proof. Fix k. Let $R_k(t) \triangleq \mathsf{refine}^{f_k, g_k}(t)$. We first show (f_k, g_k) is sound for $P_{\mathsf{typeof}=k}$ (Definition 22). Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\mathsf{typeof}=k}$. We distinguish cases on k.

- If $k \in \{\texttt{"string"}, \texttt{"number"}, \texttt{"bigint"}, \texttt{"boolean"}\}$ then $P_{\mathsf{typeof}=k}$ is a single tag-class, so by A0 we have $\mathsf{tag}(s) = \mathsf{rtag}(v)$, hence $f_k(s) = \mathsf{true}$ by definition of f_k. Since $g_k \triangleq g_{\mathrm{id}}$ is the identity, $v \in [\![g_k(s)]\!]$ is immediate.

- If $k = \texttt{"undefined"}$, then $v \in P_{\mathsf{typeof}=\texttt{"undefined"}}$ means $\mathsf{rtag}(v) \in \{0, 1\}$, and by A0 $\mathsf{tag}(s) \in \{0, 1\}$, hence $f_k(s) = \mathsf{true}$; again $v \in [\![g_k(s)]\!]$ is immediate since g_k is the identity.

- If $k = \texttt{"object"}$, then $v \in P_{\mathsf{typeof}=\texttt{"object"}}$ implies $\mathsf{rtag}(v) \in \{2, 9, 10\}$ by definition, so by A0 we have $\mathsf{tag}(s) \in \{2, 9, 10\}$, hence $f_k(s) = \mathsf{true}$ by definition of $f_{\texttt{"object"}}$. Again $v \in [\![g_k(s)]\!]$ is immediate since g_k is the identity.

- If $k = \texttt{"function"}$, then $v \in P_{\mathsf{typeof}=\texttt{"function"}}$ means $v \in \mathsf{ObjVal}$ and $\mathsf{callable}(v) = \mathsf{true}$. By A0, $\mathsf{tag}(s) = \mathsf{rtag}(v) = 9$, so $s = \texttt{object}(F)$ for some F. By Definition 7, $v \in [\![\texttt{object}(F)]\!]$ and $\mathsf{callable}(v) = \mathsf{true}$ implies $F_{\mathrm{call}} \neq \mathsf{None}$ (since None requires $\mathsf{callable}(v) = \mathsf{false}$). Thus $F_{\mathrm{call}} \in \{\mathsf{Definite}(\kappa), \mathsf{Maybe}(\kappa)\}$ for some κ, so $f_{\texttt{"function"}}(s) = \mathsf{true}$.

  For the mapper, we show $v \in [\![g_{\texttt{"function"}}(s)]\!]$:

  - If $F_{\mathrm{call}} = \mathsf{Definite}(\kappa)$, then $g_{\texttt{"function"}}(s) = s$, so $v \in [\![s]\!]$ immediately.
  - If $F_{\mathrm{call}} = \mathsf{Maybe}(\kappa)$, then $g_{\texttt{"function"}}(s) = \texttt{object}(F')$ with $F'_{\mathrm{call}} = \mathsf{Definite}(\kappa)$. Since $\mathsf{callable}(v) = \mathsf{true}$, the constraint for $\mathsf{Definite}(\kappa)$ is satisfied. The field constraints are unchanged, so $v \in [\![\texttt{object}(F')]\!]$.

Thus (f_k, g_k) is sound and by Lemma 25, R_k is predicate-sound for $P_{\mathsf{typeof}=k}$.

False branch. For most k, the false-branch uses $(\neg f_k, g_{\mathrm{id}})$, and soundness follows analogously. For $k = \texttt{"function"}$, we use $(f_{\neg\texttt{"function"}}, g_{\neg\texttt{"function"}})$. Let $v \in \neg P_{\mathsf{typeof}=\texttt{"function"}}$ and $v \in [\![s]\!]$. If s is not an object type, $f_{\neg\texttt{"function"}}(s) = \mathsf{true}$ and $v \in [\![g_{\neg\texttt{"function"}}(s)]\!] = [\![s]\!]$. If $s = \texttt{object}(F)$, then $v \in \mathsf{ObjVal}$ (by A0) and $\mathsf{callable}(v) = \mathsf{false}$ (since $v \notin P_{\mathsf{typeof}=\texttt{"function"}}$). By Definition 7, $F_{\mathrm{call}} \neq \mathsf{Definite}(\kappa)$ (since that would require $\mathsf{callable}(v) = \mathsf{true}$). Thus $F_{\mathrm{call}} \in \{\mathsf{None}, \mathsf{Maybe}(\kappa)\}$, so $f_{\neg\texttt{"function"}}(s) = \mathsf{true}$. For the mapper:

- If $F_{\mathrm{call}} = \mathsf{None}$, then $g_{\neg\texttt{"function"}}(s) = s$, so $v \in [\![s]\!]$.

- If $F_{\mathrm{call}} = \mathsf{Maybe}(\kappa)$, then $g_{\neg\texttt{"function"}}(s) = \texttt{object}(F')$ with $F'_{\mathrm{call}} = \mathsf{None}$. Since $\mathsf{callable}(v) = \mathsf{false}$, the constraint for None is satisfied. The field constraints are unchanged, so $v \in [\![\texttt{object}(F')]\!]$.

Defaults. If $d_k = \mathsf{Some}(s_k)$ and $P_{\mathsf{typeof}=k} \subseteq [\![\mathsf{single}(s_k)]\!]$, then by Lemma 26, $\mathsf{def}(R_k, d_k)$ is predicate-sound for $P_{\mathsf{typeof}=k}$. □

5.5 Refinement Case Study: Algebraic Tag Equality

This corresponds to the implementation case where a discriminant field (written as `_tag` here) is compared to a string literal, and the checker refines algebraic types by filtering the set of possible constructor tags.

Definition 39 (Algebraic-Tag Predicate)**.** *Fix $\ell \in \mathsf{String}$ (a tag label) and define:*

$$P_{\mathsf{algTag}=\ell} \triangleq \{v \in \mathsf{Val} \mid v \in \mathsf{AlgVal} \wedge \mathsf{atag}(v) = \ell\}.$$

Definition 40 (Algebraic Tag Filter/Map Pairs)**.** *Fix $\ell \in \mathsf{String}$. Let $f_\ell^{\mathsf{alg}}(s) = \mathsf{true}$ for all s (the implementation does not drop non-algebraic alternatives for this refinement). Define mappers:*

- $g_\ell^{\mathsf{alg,true}}(\texttt{algebraic}(T)) \triangleq \texttt{algebraic}(T \cap \{\ell\})$*, and* $g_\ell^{\mathsf{alg,true}}(s) \triangleq s$ *for all other* s*.*
- $g_\ell^{\mathsf{alg,false}}(\texttt{algebraic}(T)) \triangleq \texttt{algebraic}(T \setminus \{\ell\})$*, and* $g_\ell^{\mathsf{alg,false}}(s) \triangleq s$ *for all other* s*.*

Lemma 41 (Algebraic Tag Equality is Predicate-Sound)**.** *For every $\ell \in \mathsf{String}$:*

$$\mathsf{refine}^{f_\ell^{\mathsf{alg}}, g_\ell^{\mathsf{alg,true}}} \text{ is predicate-sound for } P_{\mathsf{algTag}=\ell},$$
$$\mathsf{refine}^{f_\ell^{\mathsf{alg}}, g_\ell^{\mathsf{alg,false}}} \text{ is predicate-sound for } \neg P_{\mathsf{algTag}=\ell}.$$

Proof. For the true branch, we show $(f_\ell^{\mathsf{alg}}, g_\ell^{\mathsf{alg,true}})$ is sound for $P_{\mathsf{algTag}=\ell}$ (Definition 22) and apply Lemma 25. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\mathsf{algTag}=\ell}$. Then $v \in \mathsf{AlgVal}$, so $\mathsf{rtag}(v) = 10$. By A0, $\mathsf{tag}(s) = 10$, hence s must be of the form $\texttt{algebraic}(T)$ for some finite T. By Definition 8, $v \in [\![\texttt{algebraic}(T)]\!]$ implies $\mathsf{atag}(v) \in T$. Since $\mathsf{atag}(v) = \ell$, we have $\ell \in T$, hence $v \in [\![\texttt{algebraic}(T \cap \{\ell\})]\!] = [\![g_\ell^{\mathsf{alg,true}}(s)]\!]$. Also $f_\ell^{\mathsf{alg}}(s) = \mathsf{true}$ by definition.

For the false branch, we show $(f_\ell^{\mathsf{alg}}, g_\ell^{\mathsf{alg,false}})$ is sound for $\neg P_{\mathsf{algTag}=\ell}$. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \notin P_{\mathsf{algTag}=\ell}$. We have $f_\ell^{\mathsf{alg}}(s) = \mathsf{true}$ by definition. If $v \notin \mathsf{AlgVal}$, then $\mathsf{tag}(s) \neq 10$ by A0, so s is not an algebraic type and $g_\ell^{\mathsf{alg,false}}(s) = s$; hence $v \in [\![g_\ell^{\mathsf{alg,false}}(s)]\!]$. If $v \in \mathsf{AlgVal}$ but $\mathsf{atag}(v) \neq \ell$, then $\mathsf{tag}(s) = 10$ by A0, so $s = \texttt{algebraic}(T)$ for some finite T. By Definition 8, $\mathsf{atag}(v) \in T$. Since $\mathsf{atag}(v) \neq \ell$, we have $\mathsf{atag}(v) \in T \setminus \{\ell\}$, hence $v \in [\![\texttt{algebraic}(T \setminus \{\ell\})]\!] = [\![g_\ell^{\mathsf{alg,false}}(s)]\!]$. □

5.6 Refinement Case Study: `instanceof`

The implementation refines `instanceof` tests only on the true branch, by assigning the local variable a (possibly generic) instance type. This refinement updates the *instance* component of TType rather than filtering STypes.

Definition 42 (`instanceof` Semantics and Implementation-Modeled Refinement)**.** *Assume a set of runtime class names* Class *and a predicate family*

$$P_{\mathsf{inst}} : \mathsf{Class} \to \mathcal{P}(\mathsf{Val})$$

capturing the runtime truth of `instanceof` *for each class. We assume the type system's instance forms in* TType *include a corresponding instance type $T_{\mathsf{inst}}(C)$ (with implementation-chosen type arguments), and a distinguished array instance type T_{array}.*

Axiom (`instanceof` inclusion). *For each class C we assume:*

$$v \in P_{\mathsf{inst}}(C) \implies v \in [\![T_{\mathsf{inst}}(C)]\!].$$

Array special case. *We assume $P_{\mathsf{inst}}(\texttt{Array}) \subseteq [\![T_{\mathsf{array}}]\!]$.*

Lemma 43 (`instanceof`-True Overwrite is Predicate-Sound)**.** *For each class C, the constant transformer $R_C(t) \triangleq T_{\mathsf{inst}}(C)$ is predicate-sound for $P_{\mathsf{inst}}(C)$. Similarly, the constant transformer $R_{\mathsf{array}}(t) \triangleq T_{\mathsf{array}}$ is predicate-sound for $P_{\mathsf{inst}}(\texttt{Array})$.*

Proof. Fix C and let $t \in \mathsf{TType}$. If $v \in [\![t]\!] \cap P_{\mathsf{inst}}(C)$, then $v \in P_{\mathsf{inst}}(C)$. By the axiom in Definition 42, $v \in [\![T_{\mathsf{inst}}(C)]\!] = [\![R_C(t)]\!]$. Thus $[\![t]\!] \cap P_{\mathsf{inst}}(C) \subseteq [\![R_C(t)]\!]$. The array case is identical using $P_{\mathsf{inst}}(\texttt{Array}) \subseteq [\![T_{\mathsf{array}}]\!]$. □

6 Type Difference (Diff)

Definition 44 (Diff Soundness Properties)**.** *Let X be a set equipped with a semantic interpretation $[\![\cdot]\!] : X \to \mathcal{P}(\mathsf{Val})$. An operation $\mathsf{diff} : X \times X \to X$ is* sound *(with respect to $[\![\cdot]\!]$) if for all $x_1, x_2 \in X$, letting $x_3 = \mathsf{diff}(x_1, x_2)$, the following properties hold:*

$$\textit{(P1)} \quad [\![x_3]\!] \subseteq [\![x_1]\!] \qquad \text{(under-approximation)}$$
$$\textit{(P2)} \quad [\![x_3]\!] \cup [\![x_2]\!] \supseteq [\![x_1]\!] \qquad \text{(coverage)}$$

In this document we instantiate X with STypes*,* Clause*,* $\mathsf{Clauses}$*, and* TType*.*

Reader's guide. *These two properties imply that $[\![x_3]\!]$ is always a refinement of $[\![x_1]\!]$, and that it still covers all of $[\![x_1]\!]$ once $[\![x_2]\!]$ is added back. Equivalently (Lemma 45), $[\![x_3]\!]$ is an* over-approximation *of the true set difference $[\![x_1]\!] \setminus [\![x_2]\!]$ while staying within $[\![x_1]\!]$.*

Lemma 45 (Equivalent Characterization)**.** *For all $x_1, x_2, x_3 \in X$, (P1) $\wedge$ (P2) holds if and only if*

$$[\![x_1]\!] \setminus [\![x_2]\!] \subseteq [\![x_3]\!] \subseteq [\![x_1]\!].$$

Proof. ($\Rightarrow$) Let $v \in [\![x_1]\!] \setminus [\![x_2]\!]$. By (P2), $v \in [\![x_3]\!] \cup [\![x_2]\!]$. Since $v \notin [\![x_2]\!]$, we have $v \in [\![x_3]\!]$. Thus $[\![x_1]\!] \setminus [\![x_2]\!] \subseteq [\![x_3]\!]$. Combined with (P1): $[\![x_1]\!] \setminus [\![x_2]\!] \subseteq [\![x_3]\!] \subseteq [\![x_1]\!]$.

($\Leftarrow$) The right-hand inclusion $[\![x_3]\!] \subseteq [\![x_1]\!]$ is exactly (P1). For (P2),

$$[\![x_3]\!] \cup [\![x_2]\!] \supseteq ([\![x_1]\!] \setminus [\![x_2]\!]) \cup [\![x_2]\!] \supseteq [\![x_1]\!].$$

□

Definition 46 (Completeness)**.** *A diff operation is* complete *if:*

$$[\![\mathsf{diff}(t_1, t_2)]\!] = [\![t_1]\!] \setminus [\![t_2]\!]$$

By the equivalent characterization, (P1) $\wedge$ (P2) implies $[\![t_1]\!] \setminus [\![t_2]\!] \subseteq [\![\mathsf{diff}(t_1, t_2)]\!]$. Completeness additionally requires the reverse: $[\![\mathsf{diff}(t_1, t_2)]\!] \subseteq [\![t_1]\!] \setminus [\![t_2]\!]$.

6.1 STypes Diff

Definition 47 (STypes Diff)**.**

$$\mathsf{diff}_{\mathsf{STypes}}(L, R) \triangleq \{\, s \in L \mid \mathsf{tag}(s) \notin \mathsf{tags}(R) \,\}.$$

Intuition. *Since an* STypes *value stores at most one representative per runtime tag, this diff only removes whole tags: it preserves exactly the tags present on the left but absent on the right.*

Lemma 48 (STypes Diff Satisfies (P1))**.** $[\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!] \subseteq [\![L]\!]$.

Proof. $\mathsf{diff}_{\mathsf{STypes}}(L, R) \subseteq L$ by definition, so:

$$[\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!] = \bigcup_{s \in \mathsf{diff}_{\mathsf{STypes}}(L,R)} [\![s]\!] \subseteq \bigcup_{s \in L} [\![s]\!] = [\![L]\!].$$

□

Definition 49 (Same-Tag Coverage Condition)**.** *For $L, R \in \mathsf{STypes}$, define:*

$$\mathsf{STC}(L, R) \iff \forall k \in \mathsf{tags}(L) \cap \mathsf{tags}(R).\ [\![L[k]]\!] \subseteq [\![R[k]]\!].$$

Lemma 50 (STypes Diff Satisfies (P2) under STC)**.** *If $\mathsf{STC}(L, R)$ holds, then $[\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!] \cup [\![R]\!] \supseteq [\![L]\!]$.*

Proof. Let $x \in [\![L]\!]$. Then $x \in [\![L[k]]\!]$ for some $k \in \mathsf{tags}(L)$.

- Case $k \notin \mathsf{tags}(R)$: Let $s = L[k]$. Then $\mathsf{tag}(s) = k$ and $s \in \mathsf{diff}_{\mathsf{STypes}}(L, R)$, so $x \in [\![s]\!] \subseteq [\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!]$.
- Case $k \in \mathsf{tags}(R)$: By STC, $[\![L[k]]\!] \subseteq [\![R[k]]\!] \subseteq [\![R]\!]$, so $x \in [\![R]\!]$.

□

Corollary 51 (STypes Diff under STC)**.** *If $\mathsf{STC}(L, R)$ holds, then (P2) holds. In particular, if $L[k] = R[k]$ for all $k \in \mathsf{tags}(L) \cap \mathsf{tags}(R)$, then (P2) holds.*

Lemma 52 (STypes Diff Completeness)**.** $\mathsf{diff}_{\mathsf{STypes}}$ *is complete (i.e., $[\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!] = [\![L]\!] \setminus [\![R]\!]$) if and only if $\mathsf{STC}(L, R)$.*

Proof. ($\Rightarrow$) Suppose completeness holds and let $k \in \mathsf{tags}(L) \cap \mathsf{tags}(R)$. Let $s_L = L[k]$ and $s_R = R[k]$. Since $k \in \mathsf{tags}(R)$, we have $s_L \notin \mathsf{diff}_{\mathsf{STypes}}(L, R)$, so by completeness $[\![s_L]\!] \subseteq [\![R]\!]$. By A1 and the distinct-tag invariant for STypes, $[\![s_L]\!]$ can only overlap with the unique same-tag alternative $[\![s_R]\!]$, hence $[\![s_L]\!] \subseteq [\![s_R]\!]$, i.e. $[\![L[k]]\!] \subseteq [\![R[k]]\!]$.

($\Leftarrow$) We show $[\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!] = [\![L]\!] \setminus [\![R]\!]$.

($\subseteq$) Let $x \in [\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!]$. Then $x \in [\![s]\!]$ for some $s \in \mathsf{diff}_{\mathsf{STypes}}(L, R)$. Let $k = \mathsf{tag}(s)$. Then $k \notin \mathsf{tags}(R)$, so by A1 no element of R has tag k, hence $x \notin [\![R]\!]$. Also $s \in L$, so $x \in [\![L]\!]$. Therefore $x \in [\![L]\!] \setminus [\![R]\!]$.

($\supseteq$) Let $x \in [\![L]\!] \setminus [\![R]\!]$. Then $x \in [\![L[k]]\!]$ for some $k \in \mathsf{tags}(L)$. If $k \in \mathsf{tags}(R)$, then by hypothesis $[\![L[k]]\!] \subseteq [\![R[k]]\!] \subseteq [\![R]\!]$, contradicting $x \notin [\![R]\!]$. So $k \notin \mathsf{tags}(R)$, hence $L[k] \in \mathsf{diff}_{\mathsf{STypes}}(L, R)$ and $x \in [\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!]$. □

Corollary 53 (When STypes Diff is Incomplete)**.** $\mathsf{diff}_{\mathsf{STypes}}(L, R)$ *is incomplete when there exists $k \in \mathsf{tags}(L) \cap \mathsf{tags}(R)$ with $[\![L[k]]\!] \not\subseteq [\![R[k]]\!]$. Example:* $\mathsf{diff}_{\mathsf{STypes}}(\{\texttt{number}\}, \{\texttt{1}\}) = \emptyset$*, but $[\![\texttt{number}]\!] \setminus [\![\texttt{1}]\!] \neq \emptyset$.*

6.2 TInsts Diff

Definition 54 (TInsts Diff)**.**

$$\mathsf{diff}_{\mathsf{TInsts}}(L, R) = L \setminus R = \{i \in L \mid i \notin R\}$$

Note. *In the implementation,* TInsts *is represented as a collection of instance AST nodes and* $\mathsf{diff}_{\mathsf{TInsts}}$ *removes elements based on* structural equality *of instances. In this paper, that corresponds exactly to the usual mathematical set difference on the set* TInst *(i.e. membership/equality is the syntactic identity of instances, not semantic overlap).*

Lemma 55 (TInsts Diff Satisfies (P1) and (P2))**.** $[\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] \subseteq [\![L]\!]$ *and* $[\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] \cup [\![R]\!] \supseteq [\![L]\!]$.

Proof. (P1): $\mathsf{diff}_{\mathsf{TInsts}}(L, R) \subseteq L$, so $[\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] \subseteq [\![L]\!]$.

(P2): Let $x \in [\![L]\!]$. Then $x \in [\![i]\!]$ for some $i \in L$.

- If $i \in R$: Then $x \in [\![i]\!] \subseteq [\![R]\!]$.
- If $i \notin R$: Then $i \in \mathsf{diff}_{\mathsf{TInsts}}(L, R)$, so $x \in [\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!]$.

□

Lemma 56 (TInsts Diff Completeness)**.** $\mathsf{diff}_{\mathsf{TInsts}}$ *is complete if and only if for all* $i \in L \setminus R$ *and* $j \in R$*:* $[\![i]\!] \cap [\![j]\!] = \emptyset$.

Proof. ($\Rightarrow$) Suppose completeness and let $i \in L \setminus R$ and $j \in R$. If $x \in [\![i]\!] \cap [\![j]\!]$, then $i \in \mathsf{diff}_{\mathsf{TInsts}}(L, R)$, so $x \in [\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] = [\![L]\!] \setminus [\![R]\!]$. But $x \in [\![j]\!] \subseteq [\![R]\!]$, contradiction.

($\Leftarrow$) We show $[\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] = [\![L]\!] \setminus [\![R]\!]$.

($\subseteq$) Let $x \in [\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!]$. Then $x \in [\![i]\!]$ for some $i \in L \setminus R$. For any $j \in R$, $[\![i]\!] \cap [\![j]\!] = \emptyset$ by hypothesis. Thus $x \notin [\![R]\!]$, so $x \in [\![L]\!] \setminus [\![R]\!]$.

($\supseteq$) Let $x \in [\![L]\!] \setminus [\![R]\!]$. Then $x \in [\![i]\!]$ for some $i \in L$. If $i \in R$, then $x \in [\![R]\!]$, contradiction. So $i \in L \setminus R = \mathsf{diff}_{\mathsf{TInsts}}(L, R)$. □

6.3 Clause Diff

Definition 57 (Clause Diff)**.** *For clauses* $c_1 = (\mathsf{vars}_1, \mathsf{stypes}_1)$ *and* $c_2 = (\mathsf{vars}_2, \mathsf{stypes}_2)$*:*

$$\mathsf{diff}_{\mathsf{Clause}}(c_1, c_2) = \begin{cases} \bot & \text{if } \mathsf{stypes}_2 = \mathsf{None} \\ c_1 & \text{if } \mathsf{stypes}_1 = \mathsf{None} \\ (\mathsf{vars}_1, \mathsf{Some}(\mathsf{diff}_{\mathsf{STypes}}(S_1, S_2))) & \text{if } \mathsf{stypes}_1 = \mathsf{Some}(S_1), \mathsf{stypes}_2 = \mathsf{Some}(S_2) \end{cases}$$

where $\bot$ *represents the empty type (dropped from results) with* $[\![\bot]\!] = \emptyset$.

Lemma 58 (Clause Diff Satisfies (P1))**.** $[\![\mathsf{diff}_{\mathsf{Clause}}(c_1, c_2)]\!] \subseteq [\![c_1]\!]$.

Proof. Case analysis:

- $\mathsf{stypes}_2 = \mathsf{None}$: Result is $\bot$. $[\![\bot]\!] = \emptyset \subseteq [\![c_1]\!]$.
- $\mathsf{stypes}_1 = \mathsf{None}$: Result is c_1. $[\![c_1]\!] \subseteq [\![c_1]\!]$.
- Both Some:

$$\begin{aligned} [\![(\mathsf{vars}_1, \mathsf{Some}(\mathsf{diff}_{\mathsf{STypes}}(S_1, S_2)))]\!] &= \left(\textstyle\bigcap_{v \in \mathsf{vars}_1} [\![v]\!]\right) \cap [\![\mathsf{diff}_{\mathsf{STypes}}(S_1, S_2)]\!] \\ &\subseteq \left(\textstyle\bigcap_{v \in \mathsf{vars}_1} [\![v]\!]\right) \cap [\![S_1]\!] = [\![c_1]\!]. \end{aligned}$$

□

Lemma 59 (Clause Diff Satisfies (P2) Conditionally)**.** $\mathsf{diff}_{\mathsf{Clause}}$ *satisfies (P2) when:*

- *Case* $\mathsf{stypes}_2 = \mathsf{None}$*: Always for well-formed clauses (since then* c_2 *is the unique top clause and* $[\![c_2]\!] = \mathsf{Val} \supseteq [\![c_1]\!]$*).*
- *Case* $\mathsf{stypes}_1 = \mathsf{None}$*: Always (since result is* c_1*).*
- *Case both* Some*: When* $\mathsf{diff}_{\mathsf{STypes}}(S_1, S_2)$ *satisfies (P2).*

6.4 Clauses Diff

Definition 60 (Clauses Diff)**.** *For $C_1, C_2 \in \mathsf{Clauses}$, let $c_\emptyset = C_2(\emptyset)$ if $\emptyset \in \mathrm{dom}(C_2)$, else undefined.*

$$\mathsf{diff}_{\mathsf{Clauses}}(C_1, C_2)(k) = \begin{cases} \mathsf{diff}_{\mathsf{Clause}}(\mathsf{diff}_{\mathsf{Clause}}(C_1(k), C_2(k)), c_\emptyset) & \text{if } k \in \mathrm{dom}(C_2) \wedge c_\emptyset \text{ defined} \\ \mathsf{diff}_{\mathsf{Clause}}(C_1(k), C_2(k)) & \text{if } k \in \mathrm{dom}(C_2) \wedge c_\emptyset \text{ undefined} \\ \mathsf{diff}_{\mathsf{Clause}}(C_1(k), c_\emptyset) & \text{if } k \notin \mathrm{dom}(C_2) \wedge c_\emptyset \text{ defined} \\ C_1(k) & \text{if } k \notin \mathrm{dom}(C_2) \wedge c_\emptyset \text{ undefined} \end{cases}$$

for $k \in \mathrm{dom}(C_1)$. Clauses mapped to $\bot$ are dropped.

Note. *Whenever $k \in \mathrm{dom}(C_2)$, the call $\mathsf{diff}_{\mathsf{Clause}}(C_1(k), C_2(k))$ compares clauses stored under the* same key *k, so their variable-sets match by the* Clauses *well-formedness condition (keys agree with clause variables). The only subtraction that may use a different key is via $c_\emptyset = C_2(\emptyset)$, which is deliberately applied to every $C_1(k)$; this corresponds to the implementation behavior and explains why the right-hand clause's variables are not used by $\mathsf{diff}_{\mathsf{Clause}}$.*

Why the $\emptyset$-clause matters. *The distinguished key $\emptyset$ corresponds to clauses that do not require any type parameters. When present on the right, subtracting $c_\emptyset$ from every left clause models a "global" elimination that should apply regardless of which type-parameter constraints produced the left clause.*

Lemma 61 (Clauses Diff Satisfies (P1))**.** $[\![\mathsf{diff}_{\mathsf{Clauses}}(C_1, C_2)]\!] \subseteq [\![C_1]\!]$.

Proof. Each output clause is derived from $C_1(k)$ via $\mathsf{diff}_{\mathsf{Clause}}$, which satisfies (P1). □

6.5 TType Diff

Definition 62 (TType Diff)**.** *For $t_1 = (\mathsf{insts}_1, \mathsf{clauses}_1)$ and $t_2 = (\mathsf{insts}_2, \mathsf{clauses}_2)$:*

$$\mathsf{diff}_{\mathsf{TType}}(t_1, t_2) = (\mathsf{diff}_{\mathsf{TInsts}}(\mathsf{insts}_1, \mathsf{insts}_2), \mathsf{diff}_{\mathsf{Clauses}}(\mathsf{clauses}_1, \mathsf{clauses}_2))$$

Theorem 63 (TType Diff Satisfies (P1))**.** $[\![\mathsf{diff}_{\mathsf{TType}}(t_1, t_2)]\!] \subseteq [\![t_1]\!]$.

Proof.

$$\begin{aligned} [\![\mathsf{diff}_{\mathsf{TType}}(t_1, t_2)]\!] &= [\![\mathsf{diff}_{\mathsf{TInsts}}(\mathsf{insts}_1, \mathsf{insts}_2)]\!] \cup [\![\mathsf{diff}_{\mathsf{Clauses}}(\mathsf{clauses}_1, \mathsf{clauses}_2)]\!] \\ &\subseteq [\![\mathsf{insts}_1]\!] \cup [\![\mathsf{clauses}_1]\!] = [\![t_1]\!] \end{aligned}$$

by Lemmas 55 and 61. □

Theorem 64 (TType Diff Satisfies (P2) Conditionally)**.** $\mathsf{diff}_{\mathsf{TType}}$ *satisfies (P2) when all underlying* $\mathsf{diff}_{\mathsf{STypes}}$ *calls satisfy (P2). In particular, (P2) holds when* STC *holds for all same-tag eliminations.*

Theorem 65 (TType Diff Completeness)**.** $\mathsf{diff}_{\mathsf{TType}}$ *is complete when:*

1. $\mathsf{diff}_{\mathsf{TInsts}}$ *is complete (structurally distinct instances are semantically disjoint), and*
2. $\mathsf{diff}_{\mathsf{STypes}}$ *is complete for all clause pairs (same-tag implies $[\![s]\!] \subseteq [\![s']\!]$).*

Corollary 66 (Explicit Completeness Conditions for $\mathsf{diff}_{\mathsf{TType}}$)**.** *Let $t_1 = (\mathsf{insts}_1, \mathsf{clauses}_1)$ and $t_2 = (\mathsf{insts}_2, \mathsf{clauses}_2)$. Then $\mathsf{diff}_{\mathsf{TType}}(t_1, t_2)$ is complete (i.e. $[\![\mathsf{diff}_{\mathsf{TType}}(t_1, t_2)]\!] = [\![t_1]\!] \setminus [\![t_2]\!]$) under the following explicit hypotheses:*

1. ***Instance disjointness:*** *For all* $i \in \mathsf{insts}_1$ *and* $j \in \mathsf{insts}_2$ *with* $i \neq j$*, we have* $[\![i]\!] \cap [\![j]\!] = \emptyset$*.*

2. ***Top-clause alignment:*** *If* $\mathsf{clauses}_1$ *is top (i.e. contains the unique top clause* $(\emptyset, \mathsf{None})$*), then* $\mathsf{clauses}_2$ *is also top.*

3. ***Same-tag coverage for clause subtractions:***

   (a) *For* $k \in \mathrm{dom}(\mathsf{clauses}_1) \cap \mathrm{dom}(\mathsf{clauses}_2)$ *with* $\mathsf{clauses}_i(k) = (k, \mathsf{Some}(S_i))$*:* $\mathsf{STC}(S_1, S_2)$*.*

   (b) *If* $\mathsf{clauses}_2(\emptyset) = (\emptyset, \mathsf{Some}(S_0))$*: for all* k *with* $\mathsf{clauses}_1(k) = (k, \mathsf{Some}(S_1))$*, we require* $\mathsf{STC}(S_1, S_0)$*.*

7 Application to Type Inference

Definition 67 (Subtyping Check with Validation)**.** *When checking* $u_1 \leq u_2$ *where* u_2 *contains type variable* T*:*

1. *Compute* $\mathsf{diff}(u_1, u_2)$ *and grow* T*'s lower bound.*

2. *Expand* u_1, u_2 *using the substitution.*

3. *Validate via* $\mathsf{isSub}(\mathit{expand}(u_1), \mathit{expand}(u_2))$*.*

Theorem 68 (Inference Soundness)**.** *If* isSub *correctly validates subtyping on expanded types, then type inference is sound regardless of whether* diff *satisfies (P2).*

Proof. The diff result is used only to guide inference (e.g. by accumulating bounds or hints about T). The acceptance/rejection decision is made solely by isSub on expanded types. Therefore, an imprecise diff can at worst cause incompleteness (e.g. by failing to discover a useful bound, leading to a later rejection), but it cannot cause an unsound acceptance when isSub is correct. □

8 Examples of Incompleteness

We present two concrete examples of incompleteness in SKJS:

- a *control-flow refinement* case that TypeScript accepts but SKJS currently fails to narrow (a missing implementation case), and
- a diff*-based* case where diff is incomplete.

8.1 Example 1: Missing Strict-Equality Refinement for Primitive Literals

TypeScript code:

```
function f(x: 1 | 2): 2 {
  if (x === 1) { return 2; }
  return x;  // x should be narrowed to 2
}
```

Mathematical setup: Both `1` and `2` have the same runtime tag 4. SKJS represents primitive literal unions by storing a finite literal-set payload inside a single tag representative, so `1 | 2` is representable and the narrowed type `2` is expressible. Unlike Example 2, this is not a representational limitation.

Incompleteness (missing implementation case): The control-flow refinement for strict equality in SKJS currently handles only:

1. algebraic discriminant comparisons (e.g. `x._tag === "Foo"`),
2. `typeof` comparisons, and
3. comparisons of a local to `null` or `undefined`.

It does *not* implement refinement for comparisons of a local against a primitive literal such as `x === 1`. Therefore, SKJS does not narrow `x` in either branch and rejects `return x;`, even though TypeScript narrows `x` to `2`.

Could it be implemented? Yes. Since `1|2` is stored as a finite literal-set at the numeric tag, the true branch could intersect the payload with {1} and the false branch could subtract {1}, yielding {2}. This requires adding a strict-equality refinement case for primitive literals, not changing the STypes normal form.

8.2 Example 2: Subtracting from Top (`unknown`)

Note on {} syntax. In TypeScript (with `--strictNullChecks`), the type {} behaves as a "non-nullish top": it includes all values except `null` and `undefined`, including primitives such as numbers and strings. In SKJS, the surface syntax {} denotes an *object* type (runtime tag 9) and does *not* include primitives. The example below uses TypeScript's interpretation of {}.

TypeScript code (requires `--strictNullChecks`):

```
function f(x: unknown): {} {
  if (x !== null && x !== undefined) {
    return x;  // x should be narrowed to {}
  }
  return {};
}
```

Mathematical setup: The type `unknown` is represented as a clause with $\mathsf{stypes} = \mathsf{None}$ (top). Let $c_1 = (\emptyset, \mathsf{None})$ represent `unknown`, and let $c_2 = (\emptyset, \mathsf{Some}(\{\texttt{undefined}, \texttt{null}\}))$ represent `null | undefined`.

Incompleteness: By definition of $\mathsf{diff}_{\mathsf{Clause}}$:

$$\mathsf{diff}_{\mathsf{Clause}}(c_1, c_2) = c_1 \quad (\text{since } \mathsf{stypes}_1 = \mathsf{None})$$

So $[\![\mathsf{diff}_{\mathsf{Clause}}(c_1, c_2)]\!] = [\![c_1]\!] = \mathsf{Val}$ (all values). But the true set difference is:

$$[\![c_1]\!] \setminus [\![c_2]\!] = \mathsf{Val} \setminus ([\![\texttt{null}]\!] \cup [\![\texttt{undefined}]\!]) \neq \mathsf{Val}$$

The result is incomplete: diff returns `unknown` instead of the narrowed non-nullish type. The clause diff cannot "subtract from top" because None does not enumerate tags—there is no finite set from which to remove the subtracted alternatives. The implementation in `Clause::diff` simply returns the left clause unchanged when $\mathsf{stypes}_1 = \mathsf{None}$.

Could it be implemented? Yes, but it requires a representation extension. To model TypeScript's non-nullish type precisely, one could add a dedicated form (e.g. a *non-nullish* flag, or an *excluded-tag* mechanism) rather than attempting to enumerate "all other tags" inside STypes. The current SKJS implementation chooses neither approach and leaves top unchanged when subtracting a non-top right-hand side.

Making this example work in SKJS. To express this pattern in SKJS, one would need a surface spelling for non-nullish types (such as `NonNullable<T>`) and an internal representation that can express "remove null/undefined" from an arbitrary type, including top.